

Inteligență Artificială

Bogdan Alexe

bogdan.alexe@fmi.unibuc.ro

Radu Ionescu

raducu.ionescu@gmail.com

Informatică, anul II, 2025-2026

< All teams



Inteligență Artificială - 2025-2026 F... ...

- Home page
- Class Notebook
- Classwork
- Assignments
- Grades
- Insights
- Reflect

▼ Main Channels

- General
- Învățare automată - curs
- Învățare automată - lab și sem**
- Rezolvarea problemelor și căutare - curs
- Rezolvarea problemelor și căutare - lab ...**

≡ All documents

💬 In messages

+ New ▼

↑ Upload ▼

📄 Edit in grid view

🔗 Share

🔗 Copy link

📄 Forms

New

🔗

Rezolvarea problemelor și căutare - curs > **seria24**

	Name ▼	Modified ▼	Modified By ▼
	IA-INFO-PSS-2025-2026-BONUS-KAHOO...	A few seconds ago	DUMITRU BOGDAN A
	Curs1-s24.pdf	April 29	DUMITRU BOGDAN A
	Curs1-s24.pptx	April 29	DUMITRU BOGDAN A
	Curs2-s24.pdf	May 6	DUMITRU BOGDAN A
	Curs2-s24.pptx	May 6	DUMITRU BOGDAN A
	Curs3-s24.pdf	May 13	DUMITRU BOGDAN A
	Curs3-s24.pptx	May 13	DUMITRU BOGDAN A

Recapitulare – cursul trecut

1. Căutare informată

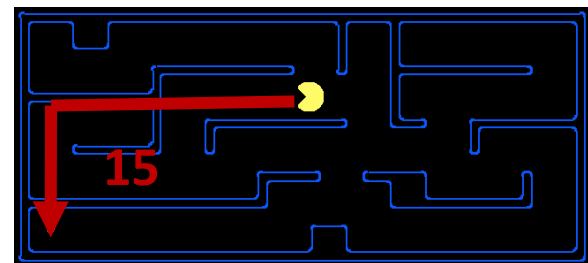
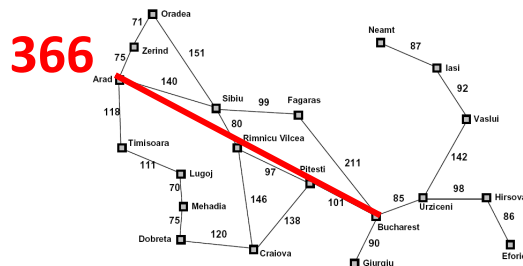
- Căutare Greedy (Greedy Best-First Search)
- Euristici admisibile
- Căutare A* (A star Best-First Search)
- Euristici consistente

Cuprinsul cursului de azi

1. Crearea de euristici admisibile
2. Proprietățile și alte variante ale algoritmului A*
3. Căutare adversarială:
 - jocuri adversariale
 - algoritmul MINIMAX
 - algoritmul α - β retezare (α - β pruning)

Crearea de euristici admisibile

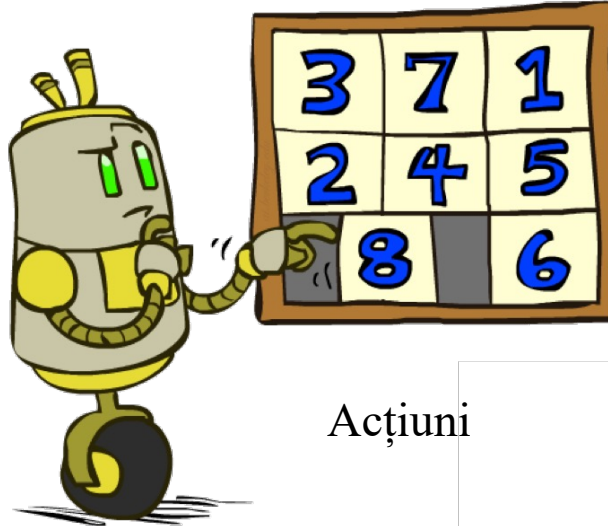
- Un ingredient fundamental în rezolvarea optimă a problemelor de căutare este găsirea unor euristici admisibile
- Deseori, euristicile admisibile sunt soluții pentru *probleme relaxate* (obținute din problema inițială prin relaxarea unor condiții), când sunt disponibile noi acțiuni



Exemplu – 8 Puzzle

7	2	4
5		6
8	3	1

Starea inițială

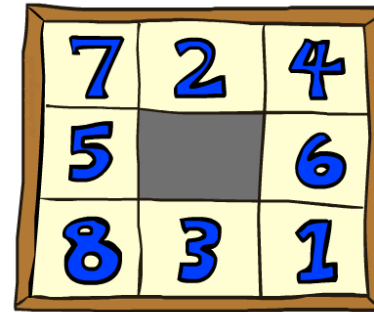


	1	2
3	4	5
6	7	8

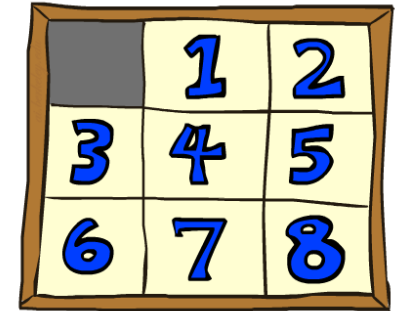
Starea finală

Exemplu – 8 Puzzle

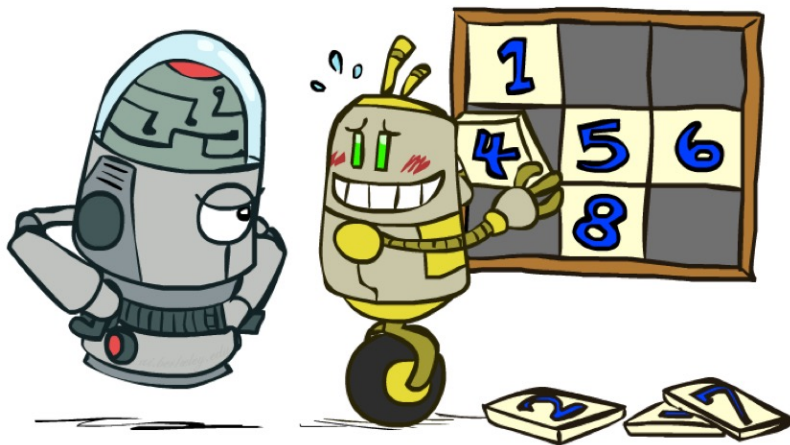
- Euristică: numărul de piese puse greșit
- Este euristică admisibilă?
- $h(\text{stare inițială}) = 8$
- Aceasta este o euristică pentru o *problema relaxată*



Starea inițială



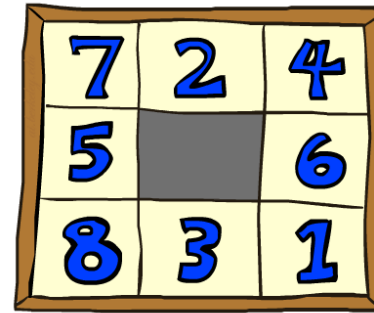
Starea finală



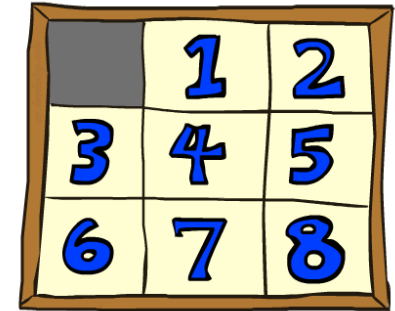
Numărul de noduri explorate când soluția optimă are...			
	...4 pași	...8 pași	...12 pași
UCS	112	6,300	3.6×10^6
A* + #TILES	13	39	227

Exemplu – 8 Puzzle

- Dacă considerăm problema 8-puzzle modificată astfel încât fiecare piesă poate fi glisată stânga/dreapta/sus/jos ignorând celelalte piese?



Starea inițială



Starea finală

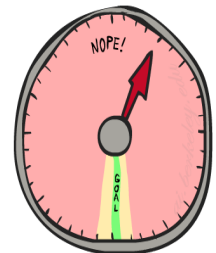
- Distanța *Manhattan*
- Este euristică admisibilă?

- $h(\text{stare inițială}) = 3 + 1 + 2 + \dots = 18$

Numărul de noduri explorate când soluția optima are...			
	...4 pași	...8 pași	...12 pași
A* + #TILES	13	39	227
A* + MANHATTAN	12	25	73

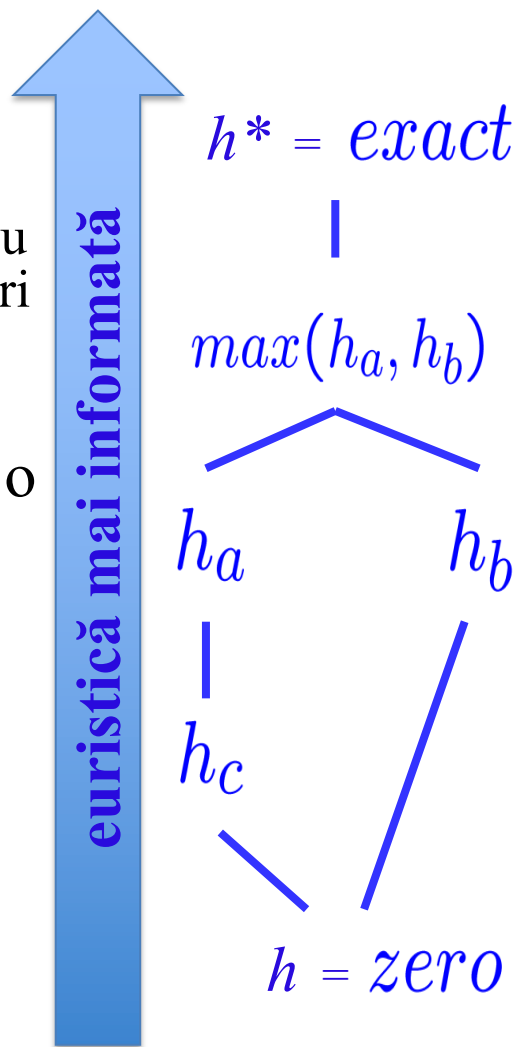
Euristici pentru A^*

- de ce să nu folosim *costul real* ca euristică $h(n) = h^*(n)$?
 - este o asemenea euristică admisibilă?
 - da (este chiar perfectă)
 - explorăm mai multe sau mai puține noduri?
 - număr minim posibil (doar nodurile de pe drumul optim)
 - există vreun dezavantaj?
 - ! pentru a calcula $h^*(n)$ trebuie să rezolvăm problema
 - ! cost computațional prohibitiv!
- compromis calitatea euristicii vs. eficiență:
 - euristică mai informată (mai aproape de costul real)
 - mai puține noduri expandate
 - ! cost mai mare de calcul
 - euristică mai simplă
 - calcul rapid
 - ! mai multe noduri explorate



Euristici dominante și banale

- Euristică dominantă: spunem că h_a domină pe h_c (scriem $h_a \geq h_c$) dacă $\forall n : h_a(n) \geq h_c(n)$
 - dacă h_a și h_c sunt admisibile și h_a domină h_c atunci A^* cu h_a va expanda mai puține sau cel mult la fel de multe noduri ca A^* cu h_c
- maximum a două euristici admisibile h_a și h_b este o euristică admisibilă $h(n) = \max(h_a(n), h_b(n))$
 - este mai informată decât fiecare în parte
- euristici banale:
 - euristica zero $h(n) = 0$
 - conduce în A^* la UCS
 - euristica exactă $h(n) = h^*(n)$
 - A^* explorează doar drumul optim
 - **!** imposibil de calculat în practică



Factorul de ramificare efectiv b^*

- O posibilă măsură de a cuantifica calitatea unei euristici h (cât de bună e, cât de mult ne ajută să rezolvăm o problemă) este *factorul de ramificare efectiv b^** .
- Echipiez algoritmul A^* cu euristica h și îl rulez pentru o problemă particulară. Pot calcula apoi b^* astfel:
 - găsesc soluția la adâncimea d
 - generez în total N noduri
 - aflu b^* ca fiind factorul de ramificare al unui arbore care generează până la adâncimea d un număr de N noduri.

$$N + 1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d$$

- pentru $d = 5$ și $N = 52$ obținem $b^* = 1.92$
- b^* poate varia pentru probleme particulare dar de obicei are valori apropiate pentru probleme complexe.
- astfel, calculând b^* pentru o serie de probleme ne facem o idee despre cât de bună este euristica considerată.
- ideal, am vrea b^* să fie cât mai aproape de 1.

Factorul de ramificare efectiv b^*

- Considerăm pentru problema 8-puzzle cele două euristici:
 - h_1 = numărul de piese așezate greșit (#Tiles)
 - h_2 = suma distanțelor Manhattan ale pieselor (Manhattan)
- Generăm 1200 de instanțe de probleme pentru 8-puzzle astfel
 - 100 de instanțe cu soluția la $d = 2$ (am nevoie de 2 mutări să găsim soluția)
 - 100 de instanțe cu soluția la $d = 4$ (am nevoie de 4 mutări să găsim soluția)
 - 100 de instanțe cu soluția la $d = 6$ (am nevoie de 6 mutări să găsim soluția)
 - ...
 - 100 de instanțe cu soluția la $d = 24$ (am nevoie de 24 mutări să găsim soluția)
 - TOTAL = 1200 de instanțe de probleme
- Rulez A^* cu h_1 și A^* cu h_2 . Calculez b^* pentru aceste probleme.
- Adicional rulez și căutarea iterativă în adâncime (IDS)

Factorul de ramificare efectiv b^*

- Rulez A^* cu h_1 și A^* cu h_2 . Calculez b^* pentru aceste probleme.
- Adicional rulez și căutarea iterativă în adâncime (IDS)

	Search Cost (nodes generated)			Effective Branching Factor		
d	IDS	$A^*(h_1)$	$A^*(h_2)$	IDS	$A^*(h_1)$	$A^*(h_2)$
2	10	6	6	2.45	1.79	1.79
4	112	13	12	2.87	1.48	1.45
6	680	20	18	2.73	1.34	1.30
8	6384	39	25	2.80	1.33	1.24
10	47127	93	39	2.79	1.38	1.22
12	3644035	227	73	2.78	1.42	1.24
14	—	539	113	—	1.44	1.23
16	—	1301	211	—	1.45	1.25
18	—	3056	363	—	1.46	1.26
20	—	7276	676	—	1.47	1.27
22	—	18094	1219	—	1.48	1.28
24	—	39135	1641	—	1.48	1.26

Figure 3.29 Comparison of the search costs and effective branching factors for the ITERATIVE-DEEPENING-SEARCH and A^* algorithms with h_1 , h_2 . Data are averaged over 100 instances of the 8-puzzle for each of various solution lengths d .

Factorul de ramificare efectiv b^*

- Rulez A^* cu h_1 și A^* cu h_2 . Calculez b^* pentru aceste probleme.
- Adicional rulez și căutarea iterativă în adâncime (IDS)
- Din definiția euristicilor avem că pentru fiecare stare $h_1(n) \leq h_2(n)$, deci h_2 domină pe h_1 . A^* cu h_1 va genera mult mai multe noduri decât A^* cu h_2 .
- Experimental, observăm că A^* cu h_2 rezolvă mai eficient decât A^* cu h_1 și mult mai eficient decât IDS. Pentru $d = 12$, A^* cu h_2 rezolvă de 50.000 de ori mai repede problema decât IDS.

Algoritmul A* ponderat

- Putem ajusta performanța lui A* prin introducerea unui parametru de pondere w . Multiplicăm euristica cu w :

$$f(n) = g(n) + w \cdot h(n)$$

- Dacă $w > 1$, atunci A* favorizează din ce în ce mai mult direcția indicată de euristică
- Dacă $w < 1$, atunci A* are din ce în ce mai puțină încredere în euristică
- Ce se întâmplă când $w \rightarrow 0$ sau $w \rightarrow \infty$?

Algoritmul A* ponderat

A*	$f(n) =$	$g(n)$	$+$	$h(n)$
UCS	$f(n) =$	$g(n)$	$+$	0
Greedy	$f(n) =$	0	$+$	$h(n)$
WA*	$f(n) =$	$g(n)$	$+$	$h(n) * w$



Proprietățile algoritmului A*

Proprietățile algoritmului A* depind în mod direct de calitatea funcției sale euristice $h(n)$ (estimarea costului până la destinație), dar și de varianta de implementare folosită: Tree Search sau Graph Search.

Caracteristică	A* tree search	A* graph search
Complexitate timp	$O(b^d)$	$O(b^d)$ – dar mult mai eficient în practică
Complexitate spațiu	$O(b^d)$ – Open List	$O(b^d)$ – Open List + Closed List
Completitudine	DA	DA
Optimalitate	DA , dacă h este admisibilă	DA , dacă h este consistentă

b : factorul de ramificare (numărul mediu de succesori ai unui nod)

d : adâncimea la care se află soluția optimă

Proprietățile algoritmului A*

Proprietățile algoritmului A* depind în mod direct de calitatea funcției sale euristice $h(n)$ (estimarea costului până la destinație), dar și de varianta de implementare folosită: Tree Search sau Graph Search.

Caracteristică	A* tree search	A* graph search
Complexitate timp	$O(b^d)$	$O(b^d)$ – dar mult mai eficient în practică
Complexitate spațiu	$O(b^d)$ – Open List	$O(b^d)$ – Open List + Closed List
Completitudine	DA	DA
Optimalitate	DA , dacă h este admisibilă	DA , dacă h este consistentă

b : factorul de ramificare (numărul mediu de succesori ai unui nod)

d : adâncimea la care se află soluția optimă

Pe probleme cu un spațiu masiv de stări, A* - Graph Search va epuiza rapid memoria RAM disponibilă înainte de a rămâne fără timp.

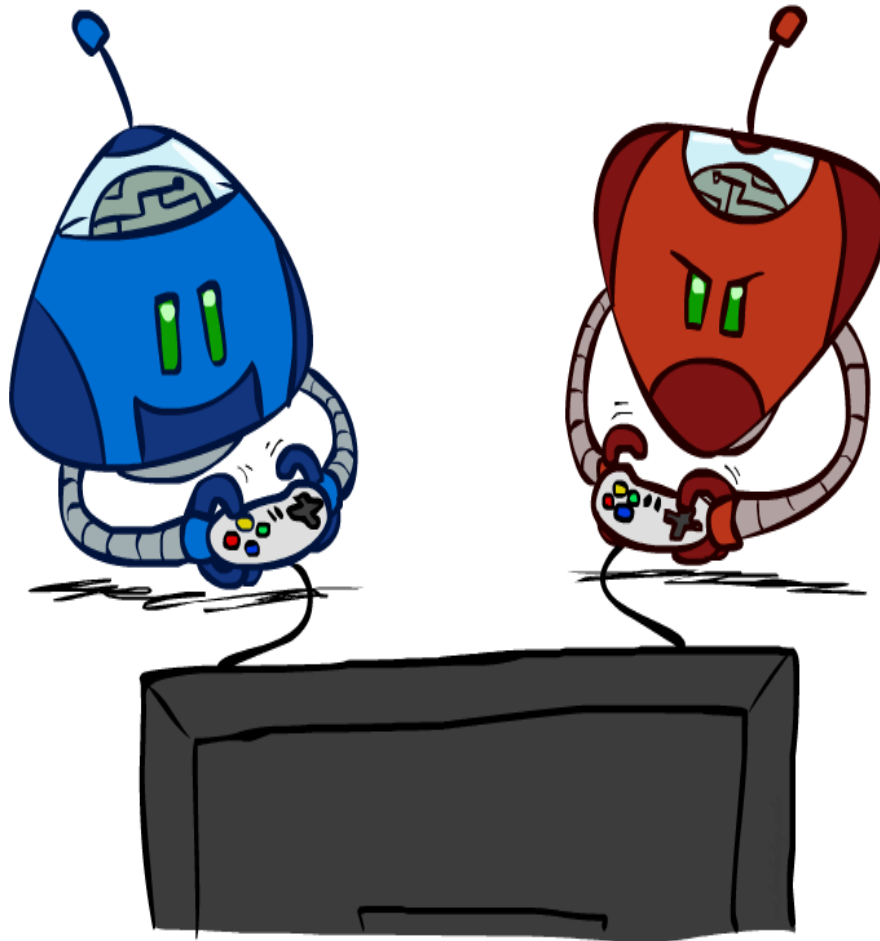
Algoritmul IDA* (iterative deepening A*)

- Combină A* (folosește $f(n) = g(n) + h(n)$) cu căutarea în adâncime iterativă
 - folosește la fiecare pas o căutare în adâncime cu limita dată de f (nu de adâncimea d)
- 1. Setăm pragul inițial: $\text{threshold} = f(\text{start}) = h(\text{start})$
- 2. Rulăm căutare în adâncime (DF), dar expandăm doar noduri cu $f(n) \leq \text{threshold}$
- 3. Dacă depășim pragul: reținem cel mai mic $f(n)$ care a depășit
- 4. Creștem pragul: $\text{threshold} = \min \{ f(n) > \text{threshold} \}$
- 5. Repetăm până găsim soluția

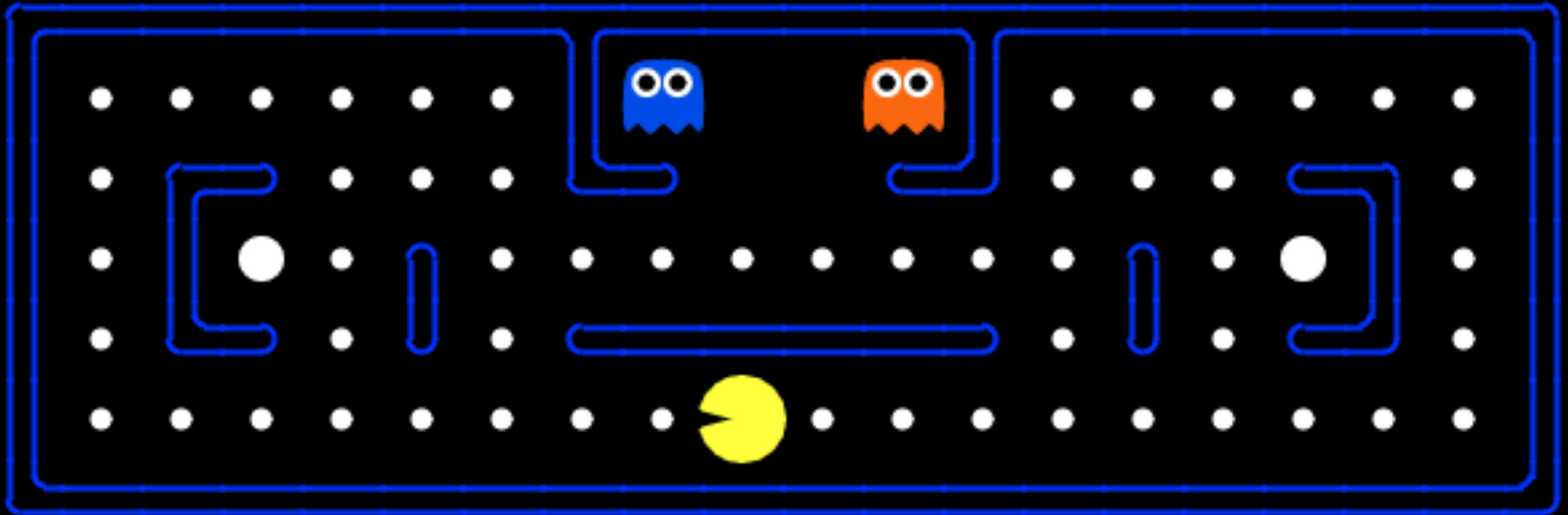
Avantaje / Dezavantaje

- ✓ memorie foarte mică (de la căutare în adâncime)
- ✓ găsește soluție optimă (dacă h este admisibilă/consistentă)
- ! re-expandează noduri (mai lent decât A*)

Căutare adversarială



Recap: Agenți inteligenți și mediile în care aceștia funcționează



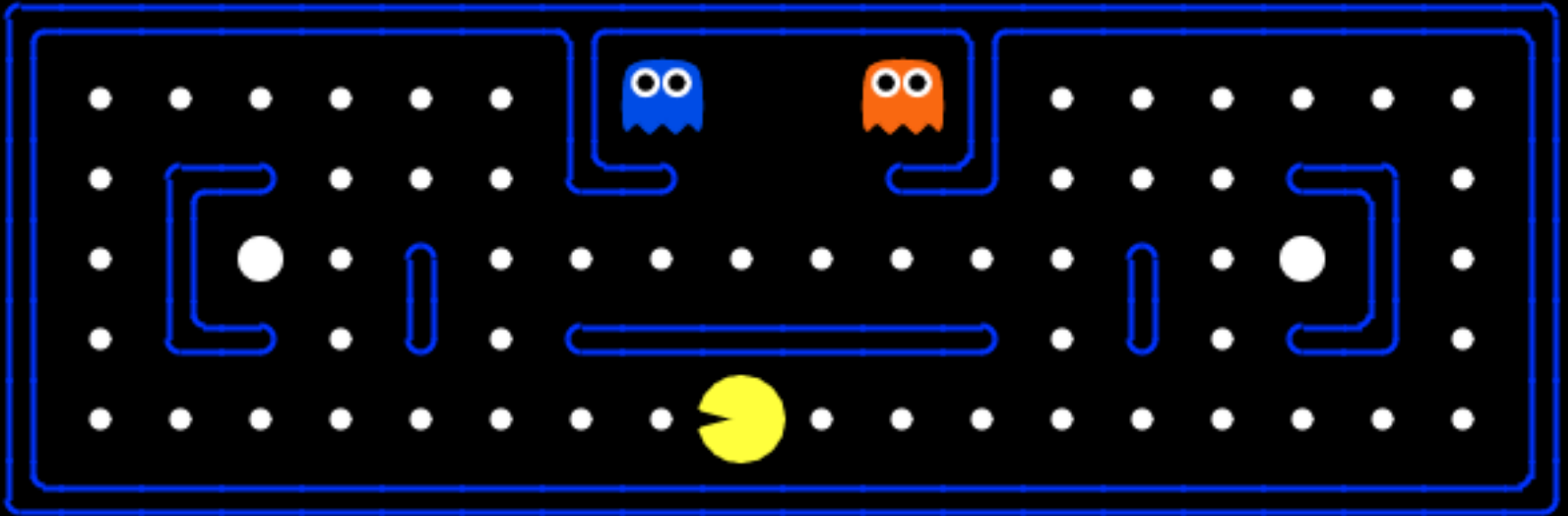
Recap: Proprietăți ale mediilor în care funcționează agenții

- **Complet observabil vs parțial observabil**: la orice moment în timp mediul este complet observabil dacă senzorii agentului îi dau acces complet la starea mediului și îi oferă toate aspectele relevante în luarea deciziei optime.
 - jocul de dame vs poker
- **Determinist vs stochastic**: dacă următoarea stare a mediului este în întregime determinată de starea curentă și de acțiunile agentului, mediul este determinist (nu există incertitudine).
 - șah vs pac-man
- **Discret vs continuu**: dacă spațiul stărilor din mediu și numărul de acțiuni ale agentului sunt finite atunci mediul este discret.
 - jocul 8-puzzle vs condusul unei mașini

Recap: Proprietăți ale mediilor în care funcționează agenții

- **Episodic vs secvențial**: într-un mediu episodic, experiența agentului este împărțită în episoade independente. Un episod constă în percepție urmată de o singură acțiune. Episoadele următoare nu depind de acțiunile episoadelor anterioare.
 - OCR pe cifre independente (ex. LeNet pe MNIST) vs șah
- **Static vs dinamic**: dacă mediul se schimbă cât timp agentul decide ce acțiuni să facă mediul este dinamic.
 - jocul de șah vs pac-man
- **Agent vs multiagent**: dacă sunt mai mulți agenți mediul este multiagent (o entitate B este considerată agent dacă are un comportament care maximizează o măsură de performanță care depinde de comportamentul (deciziile) agentului A; distingem sisteme multiagent competitive vs sisteme multiagent cooperative).
 - 8-puzzle vs șah

Recap: PAC MAN



SCORE: 0

Complet observabil vs parțial observabil
Determinist vs stocastic
Discret vs continuu
Episodic vs secvențial
Static vs dinamic
Agent vs multiagent

Complet observabil
Stocastic
Discret
Secvențial
Dinamic
Multiagent competitiv

Recap: Exemple de medii cu diverse proprietăți

Mediu	Episodic/ Secvențial	Agent / Multiagent	Determinist / Stochastic	Observabil	Static / Dinamic	Discret / Continuu
Rebus	Secvențial	Agent	Determinist	Complet	Static	Discret
Șah	Secvențial	Multiagent	Determinist	Complet	Static	Discret
Table	Secvențial	Multiagent	Stochastic	Complet	Static	Discret
Poker	Secvențial	Multiagent	Stochastic	Parțial	Static	Discret
Starcraft	Secvențial	Multiagent	Stochastic	Parțial	Dinamic	Discret
Lumea reală	Secvențial	Multiagent	Stochastic	Parțial	Dinamic	Continuu

Jocuri

Medii multi-agent:

- fiecare agent trebuie să considere acțiunile celorlalți agenți
- adversar imprevizibil, incontrolabil
- mediu competitiv

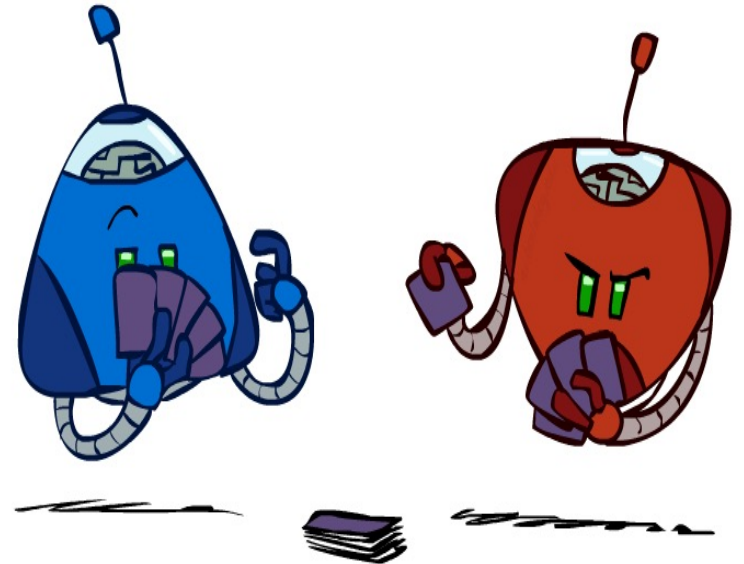
➔ Căutare adversarială = joc

Soluția este o **strategie** (policy) care specifică mutarea pentru fiecare mutare posibilă a adversarului

Problemele de căutare adversarială se deosebesc de problemele clasice de căutare prin faptul că soluția nu este o simplă secvență de acțiuni care conduce sigur la atingerea scopului, ci o **strategie** care specifică acțiunea optimă pentru fiecare posibilă mutare a adversarului.

Tipuri de jocuri

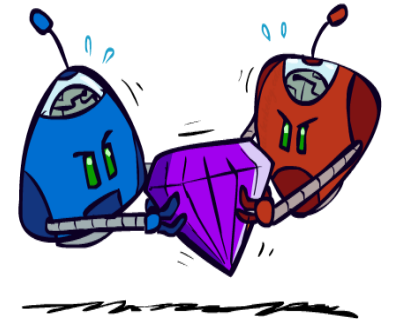
- multe tipuri de jocuri!
- dimensiuni:
 - deterministic sau stocastic?
 - unul, doi, sau mai mulți jucători?
 - joc de sumă zero?
 - complet sau parțial observabil? (se cunoaște întreaga stare)
- vrem să găsim algoritmi pentru a calcula o strategie ce recomandă o mutare din fiecare stare



Tipuri de jocuri

	determinist	stocastic
complet observabil	sah, dame, go	table, monopoly
parțial observabil	avioane, battleship	poker, catan

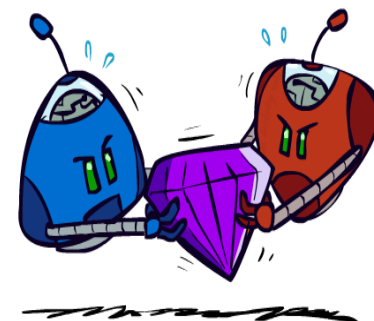
- dimensiuni:
 - deterministic sau stocastic?
 - unul, doi, sau mai mulți jucători?
 - joc de sumă zero?
 - complet sau parțial observabil? (se cunoaște întreaga stare)
- vrem să găsim algoritmi pentru a calcula o strategie ce recomandă o mutare din fiecare stare



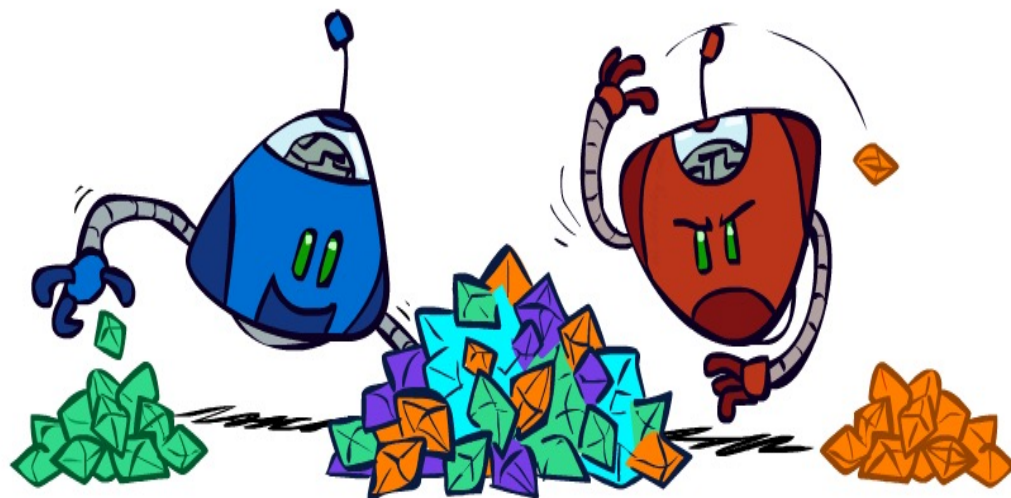
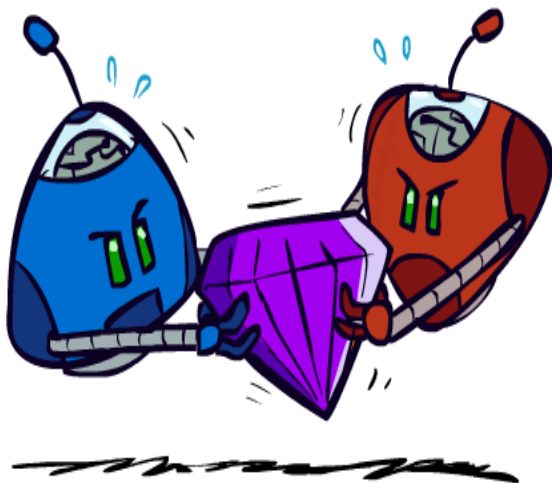
Tipuri de jocuri

	determinist	stocastic
complet observabil	sah, dame, go	table, monopoly
parțial observabil	avioane, battleship	poker, catan

- dimensiuni:
 - **deterministic** sau stocastic?
 - unul, **doi**, sau mai mulți jucători?
 - **joc de sumă zero**?
 - **complet** sau parțial **observabil**? (se cunoaște întreaga stare)
- vrem să găsim algoritmi pentru a calcula o strategie ce recomandă o mutare din fiecare stare



Jocuri cu sumă zero



- Jocuri cu sumă zero
 - agenții au utilități (evaluări ale rezultatelor) opuse
 - un agent dorește maximizarea utilității (agentul MAX), celălalt minimizarea (agentul MIN)
 - mediu adversarial pur competitiv
- Jocuri generale
 - agenții au utilități independente
 - cooperare, indiferență, competiție

Formulararea unei probleme de joc (Minimax)

- joc cu doi jucători: MAX vs. MIN
 - joc de sumă zero: câștigul unuia = pierderea celuilalt

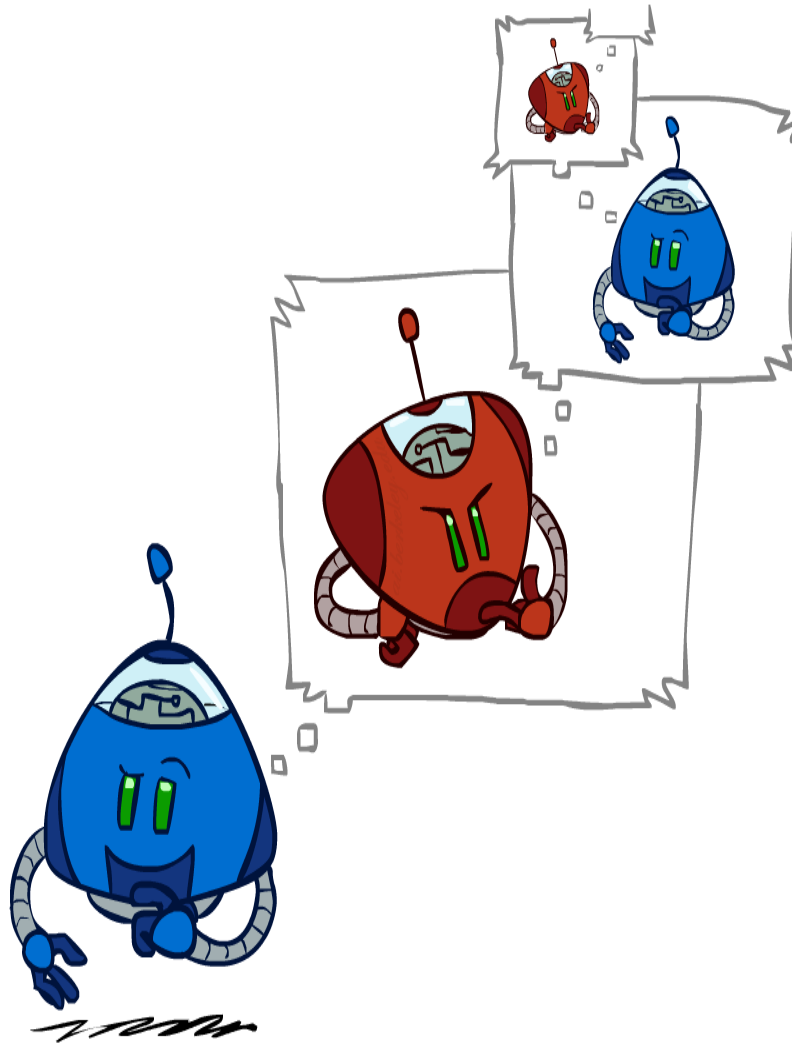
Componentele problemei

- stare inițială: s_0
- jucătorul la mutare: $\text{PLAYER}(s)$
- acțiuni legale : $\text{ACTIONS}(s)$
- modelul de tranziție (funcția succesor): $\text{RESULT}(s,a)$
- test terminal : $\text{TERMINAL-TEST}(s)$
- funcție de utilitate: $\text{UTILITY}(s)$

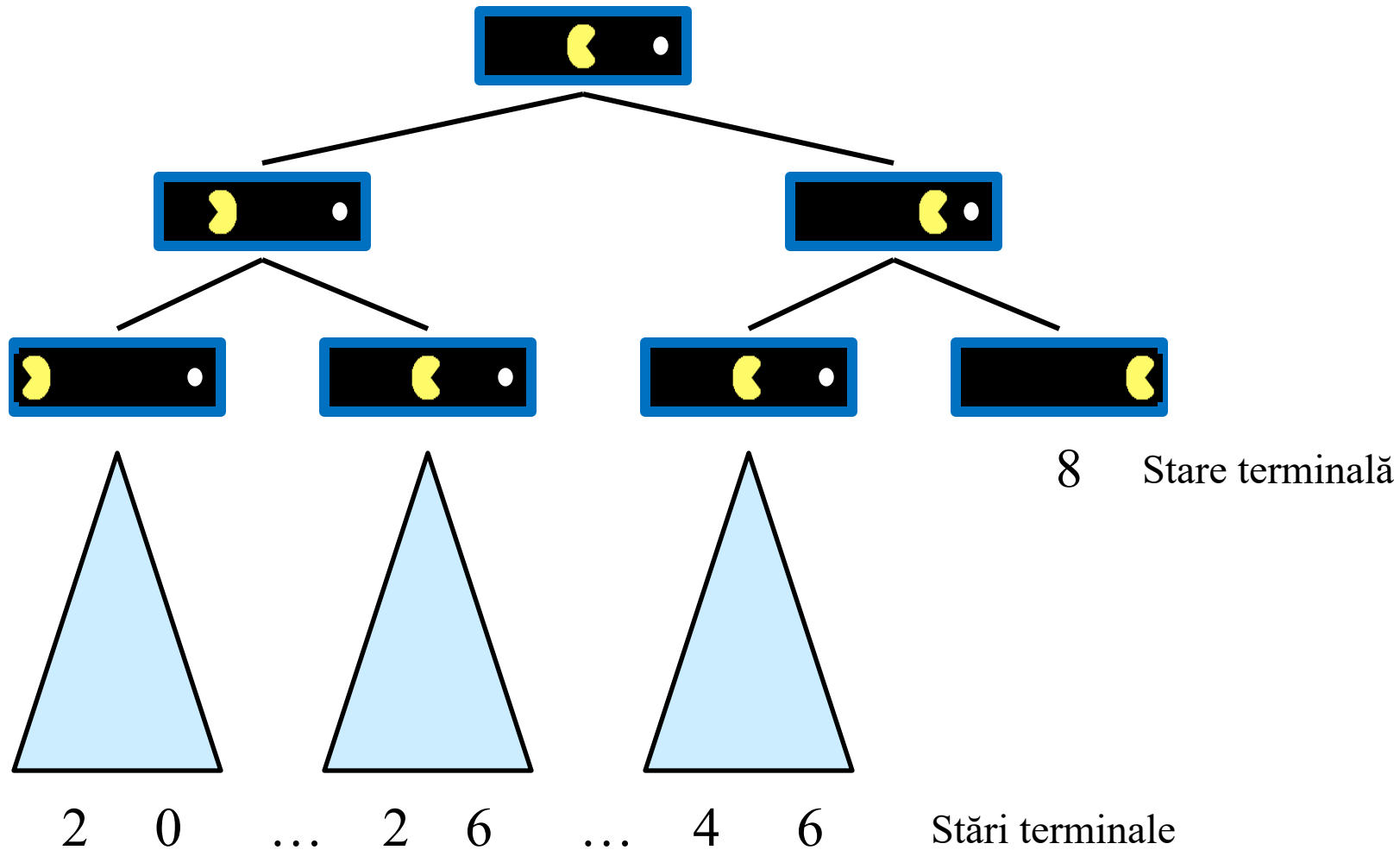
Exemplu:

$$\text{UTILITY}(s) = \begin{cases} +1 & \text{câștig (MAX)} \\ 0 & \text{remiză} \\ -1 & \text{pierdere (MAX)} \end{cases}$$

Găsirea unei strategii optime



Arbore de căutare într-un mediu cu un singur agent

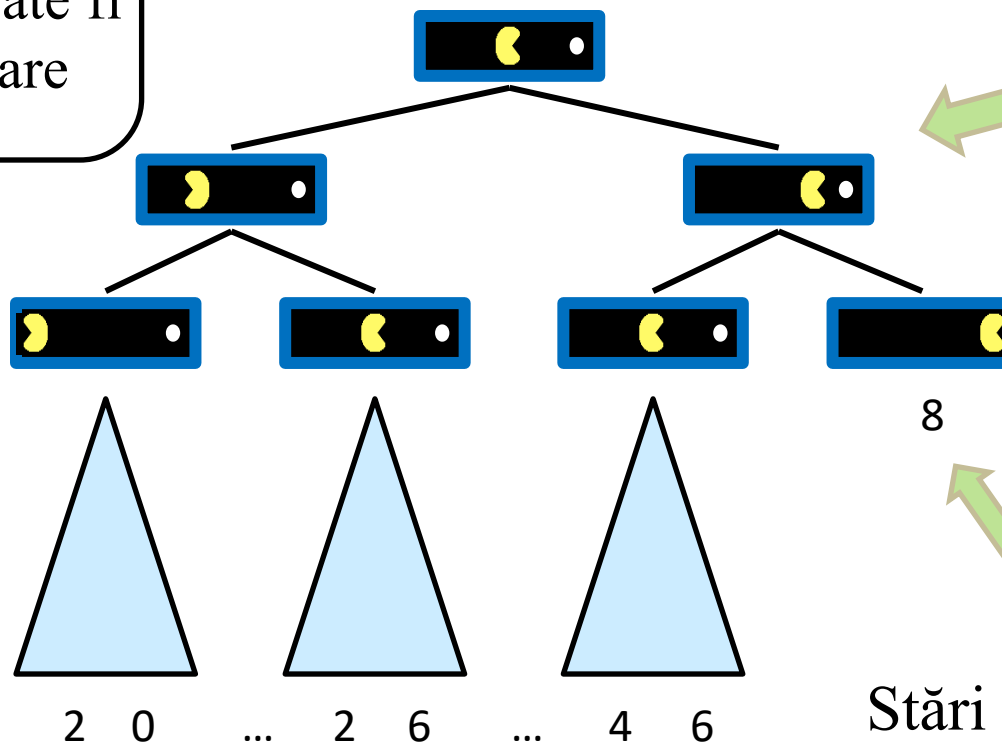


Fiecare mutare: -1 punct

Mânânc punctul alb: + 10 puncte

Valoarea unei stări

Valoarea unei stări:
cea mai bună utilitate
(rezultat) care poate fi
atinsă din acea stare



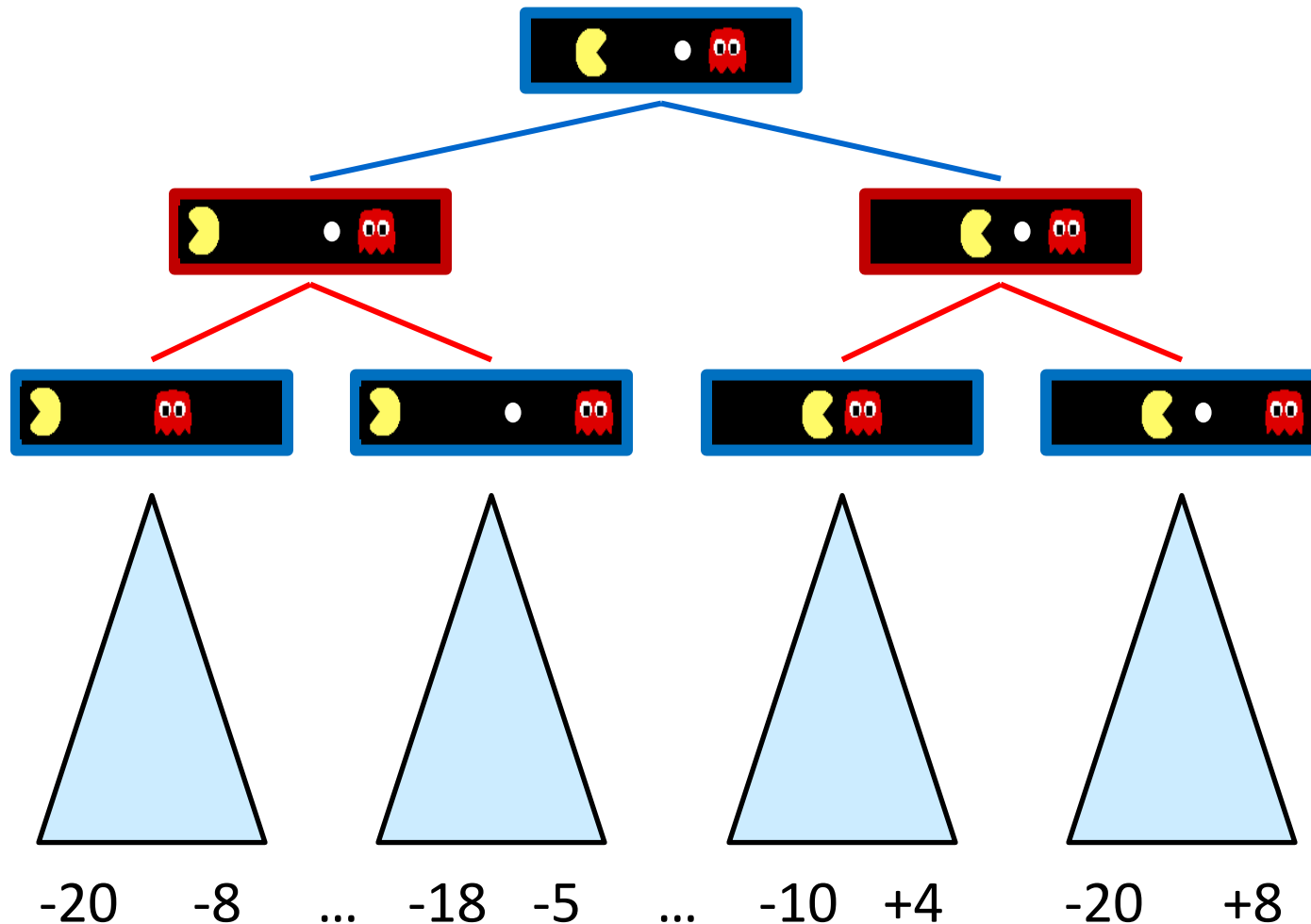
Stări neterminale:

$$V(s) = \max_{s' \in \text{Successors}(s)} V(s')$$

Stări terminale:

$V(s)$ = cunoscută, dată
de regulile jocului

Arbore de joc în medii cu doi agenți, deterministic, mutări pe rând



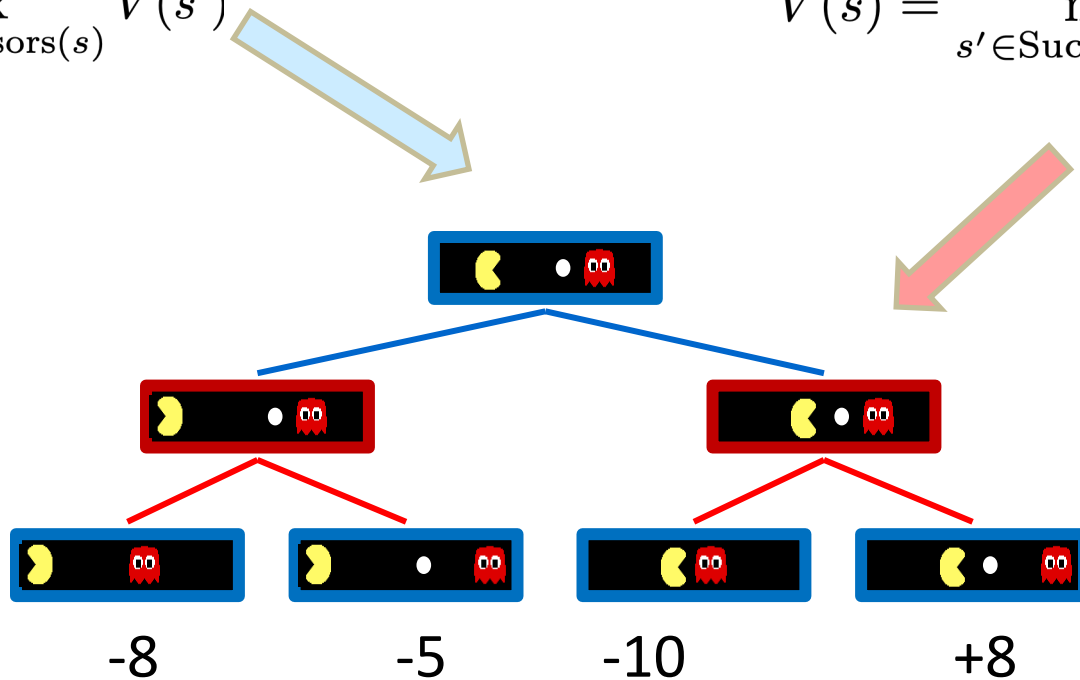
Valoare MiniMax a unei stări

Stări controlate de MAX

$$V(s) = \max_{s' \in \text{Successors}(s)} V(s')$$

Stări controlate de MIN:

$$V(s) = \min_{s' \in \text{Successors}(s)} V(s')$$

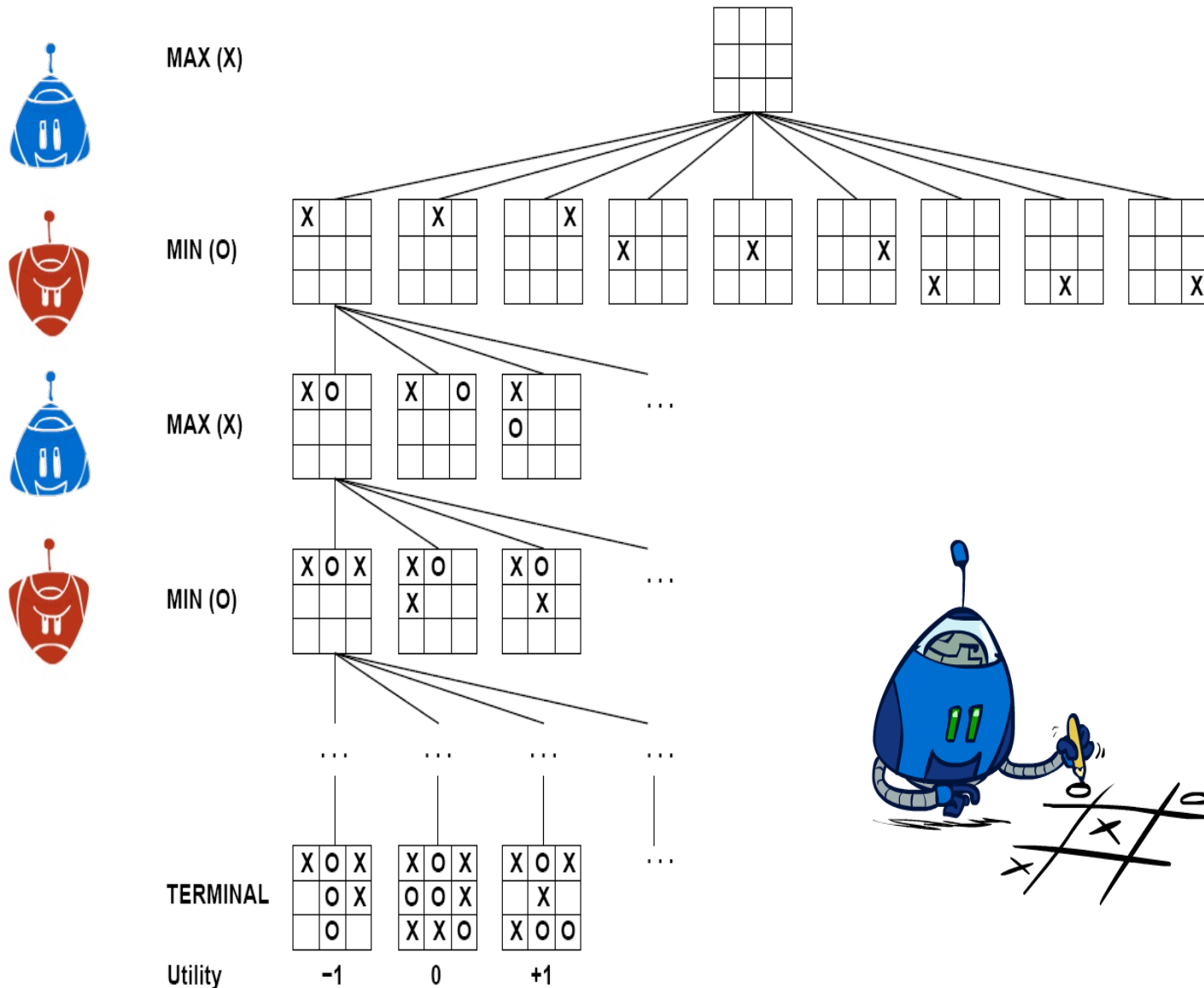


Stări terminale: $V(s)$ = cunoscută,
dată de regulile jocului

Arbore de joc

- nodurile corespund situațiilor (stărilor)
- situația inițială a jocului dată de nodul rădăcină
- arcele corespund mutărilor
- frunzele corespund situațiilor terminale (s-a terminat jocul)
- 2 jucători: MAX (de obicei este cel care face prima mutare) și MIN

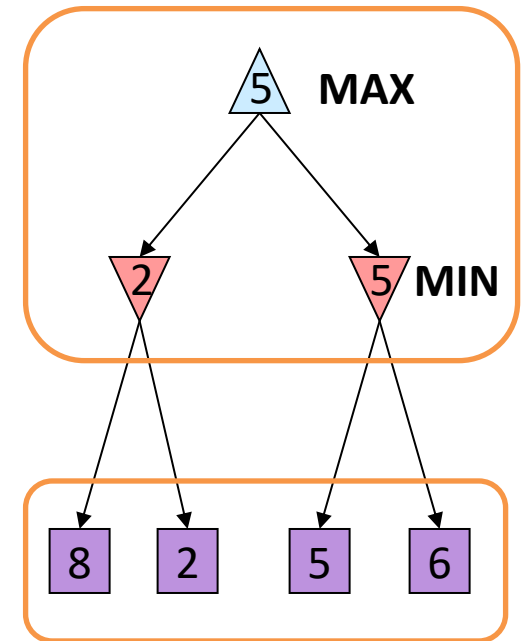
Arbore de joc pentru X și O



Algoritmul MiniMax pentru căutare adversarială

- Jocuri de sumă zero cu doi jucători, deterministe, cu informație perfectă
 - X și O, șah, dame
 - un jucător (MAX) maximizează utilitatea
 - celălalt jucător (MIN) minimizează utilitatea
- Căutare MiniMax:
 - arbore de căutare în spațiul stărilor
 - jucătorii mută pe rând
 - calculăm pentru fiecare nod valoarea **minimax**: cea mai bună utilitate care poate fi obținută împotriva unui adversar care joacă perfect
 - determină strategia optimă corespunzătoare lui MAX, care este cea mai bună primă mutare

**Valori minimax:
calculate recursiv**



**Valori terminale:
date de regulile jocului**

Algoritmul MiniMax - descriere

1. Generează întreg arborele de joc până la stările terminale
2. Aplică funcția de utilitate fiecărei stări terminale - obține valoarea stării
3. Deplasează-te înapoi în arbore, de la nodurile-frunze spre nodul-rădăcină, determinând, corespunzător fiecărui nivel al arborelui, valorile care reprezintă utilitatea nodurilor aflate la acel nivel. Propagarea acestor valori la niveluri anterioare se face prin intermediul nodurilor-părinte succesive, conform următoarei reguli:
 - dacă starea-părinte este un nod de tip MAX, atribuie-i maximul dintre valorile avute de fii săi;
 - dacă starea-părinte este un nod de tip MIN, atribuie-i minimul dintre valorile avute de fii săi;
4. Ajuns în nodul-rădăcină, alege pentru MAX acea mutare care conduce la valoarea maximă. Mutarea se numește *decizia minimax* - maximizează utilitatea, în ipoteza că oponentul joacă perfect cu scopul de a o minimiza.

Algoritmul MiniMax - formalizare

- jucătorul la mutare: $\text{PLAYER}(s)$
- acțiuni legale: $\text{ACTIONS}(s)$
- modelul de tranziție (funcția succesor): $\text{RESULT}(s,a)$
- test terminal: $\text{TERMINAL-TEST}(s)$
- funcție de utilitate: $\text{UTILITY}(s)$

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{if } \text{TERMINAL-TEST}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MIN} \end{cases}$$

Implementare MiniMax

def value(stare):

 dacă stare este o stare terminală: returnează utilitatea stării

 dacă următorul agent care mută este MAX:
 returnează max-value(stare)

 dacă următorul agent care mută este MIN: returnează min-value(stare)

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{if } \text{TERMINAL-TEST}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MIN} \end{cases}$$

Implementare MiniMax

def value(stare):

dacă stare este o stare terminală: returnează utilitatea stării

dacă următorul agent care mută este **MAX**:
returnează **max-value(stare)**

dacă următorul agent care mută este **MIN**: returnează **min-value(stare)**

def max-value(stare):

initializează $v = -\infty$

pentru fiecare successor al stării:

$v = \max(v, \text{value}(\text{successor}))$

returnează v

def min-value(stare):

initializează $v = +\infty$

pentru fiecare succesor al stării:

$v = \min(v, \text{value}(\text{successor}))$

returnează v

$$V(s) = \max_{s' \in \text{Successors}(s)} V(s')$$

$$V(s) = \min_{s' \in \text{Successors}(s)} V(s')$$

Implementare MiniMax (pe tot arborele)

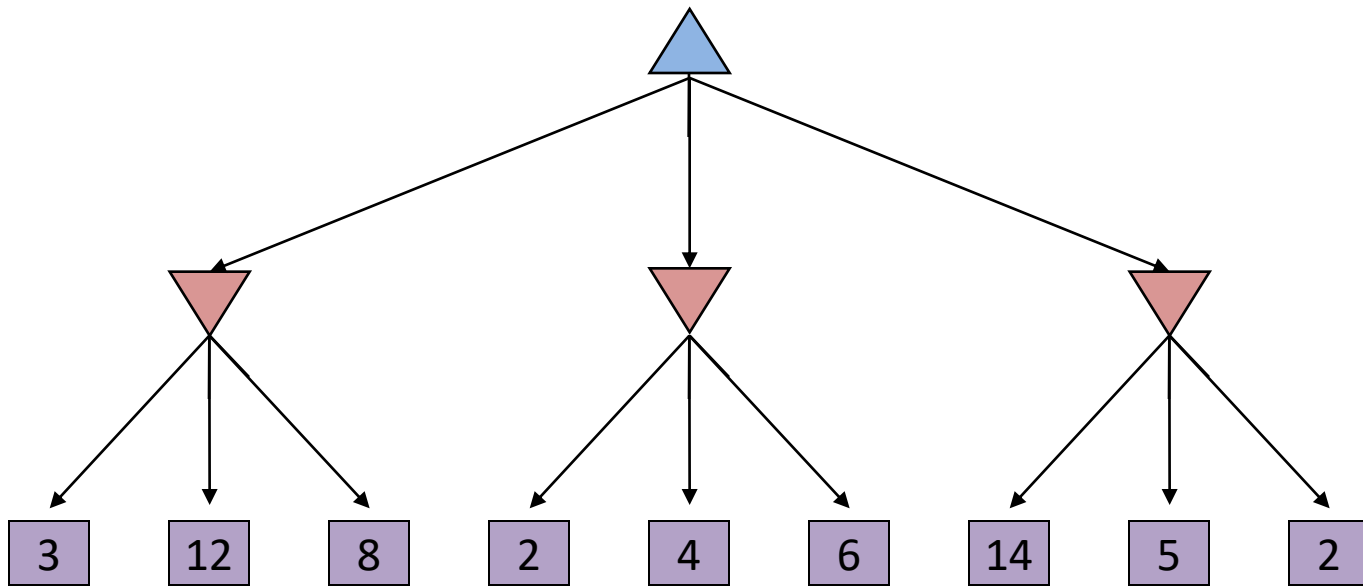
```
1. Function MaxValue(s)
2.   if (terminal(s))
3.     return eval(s)
4.   v = -infinity
5.   for (c in children(s))
6.     v' = MinValue(c)
7.     if (v' > v) v = v'
8.   return v
```

```
1. Function MinValue(s)
2.   if (terminal(s))
3.     return eval(s)
4.   v = +infinity
5.   for (c in children(s))
6.     v' = MaxValue(c)
7.     if (v' < v) v = v'
8.   return v
```

Exemplul MiniMax

MAX

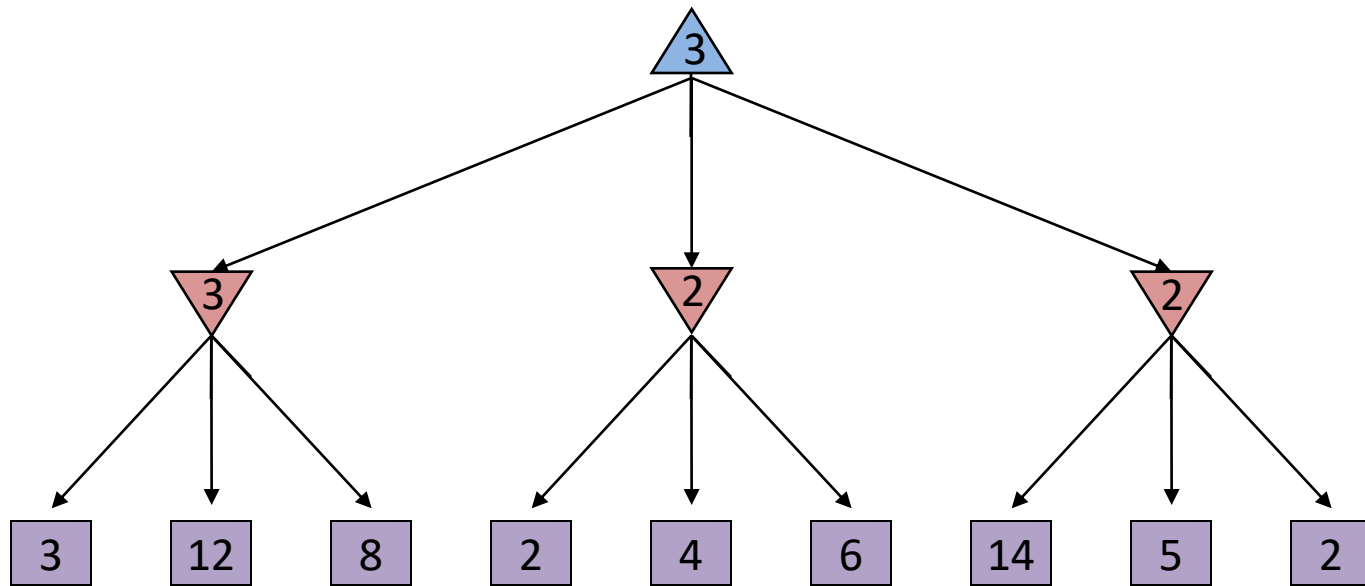
MIN



Exemplul MiniMax

MAX

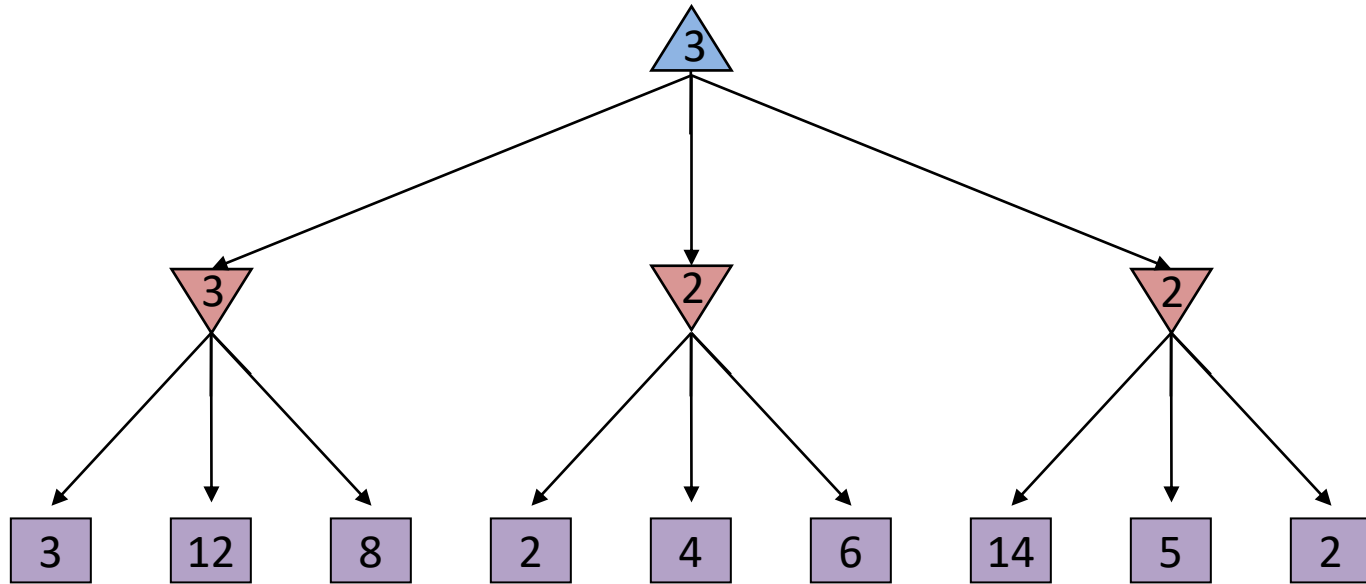
MIN



Exemplul MiniMax

MAX

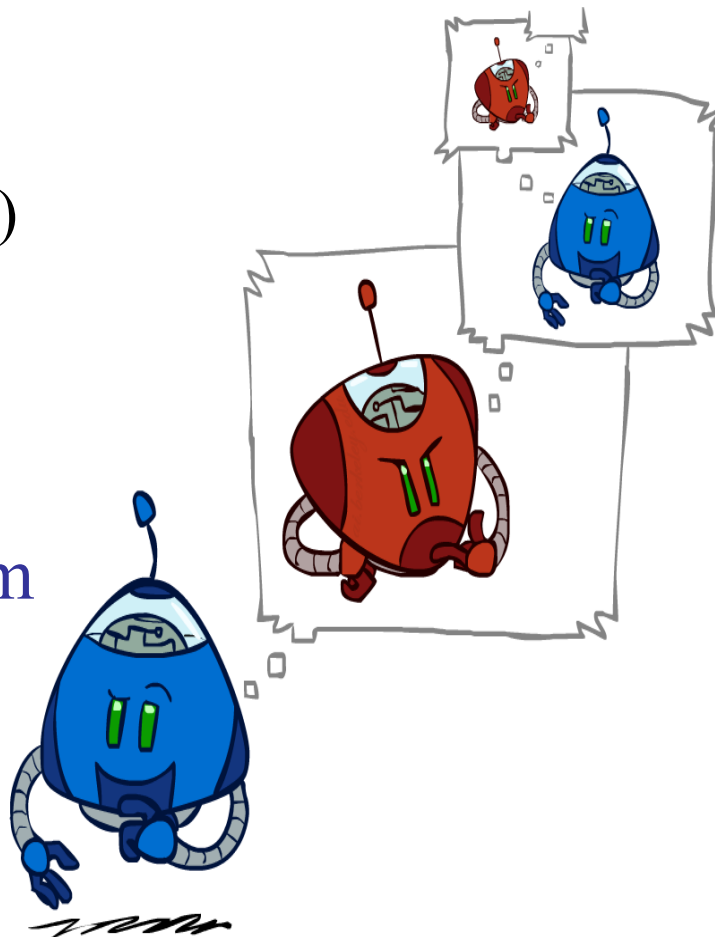
MIN



- valorile pozițiilor de la ultimul nivel sunt determinate de către funcția de utilitate și se numesc valori statice (se realizează evaluări statice).
- valorile minimax ale nodurilor interne sunt calculate în mod dinamic, în manieră bottom-up, nivel cu nivel, până când este atins nodul-rădăcină.
- valoarea rezultată este 3 și prin urmare cea mai bună mutare a lui MAX din poziția curentă este mutarea la stânga. Cel mai bun răspuns al lui MIN este mutarea la stânga. Această secvență a jocului poartă denumirea de variație principală. Ea definește jocul optim de tip minimax pentru ambele părți.
- se observă ca valoarea pozițiilor de-a lungul variației principale nu variază. Prin urmare, *mutările corecte sunt cele care conservă valoarea jocului.*

Proprietăți ale algoritmului MiniMax

- Cât de eficient este MiniMax?
 - Căutare exhaustivă în manieră depth-first (căutare în adâncime)
 - timp: $O(b^m)$
 - spațiu: $O(bm)$
- Exemple: pentru șah, $b \approx 35$, $m \approx 100$
 - o soluție exactă este imposibilă
 - dar, oare chiar trebui să explorăm întreg arborele?



Limitări date de resurse ale algoritmului MiniMax

1. **Generează întreg arborele de joc până la stările terminale**
2. Aplică funcția de utilitate fiecărei stări terminale - obține valoarea stării
3. Deplasează-te înapoi în arbore, de la nodurile-frunze spre nodul-rădăcină, determinând, corespunzător fiecărui nivel al arborelui, valorile care reprezintă utilitatea nodurilor aflate la acel nivel. Propagarea acestor valori la niveluri anterioare se face prin intermediul nodurilor-părinte succesive, conform următoarei reguli:
 - dacă starea-părinte este un nod de tip MAX, atribuie-i maximul dintre valorile avute de fii săi;
 - dacă starea-părinte este un nod de tip MIN, atribuie-i minimul dintre valorile avute de fii săi;
4. Ajuns în nodul-rădăcină, alege pentru MAX acea mutare care conduce la valoarea maximă. Mutarea se numește ***decizia minimax*** - maximizează utilitatea, în ipoteza că oponentul joacă perfect cu scopul de a o minimiza.

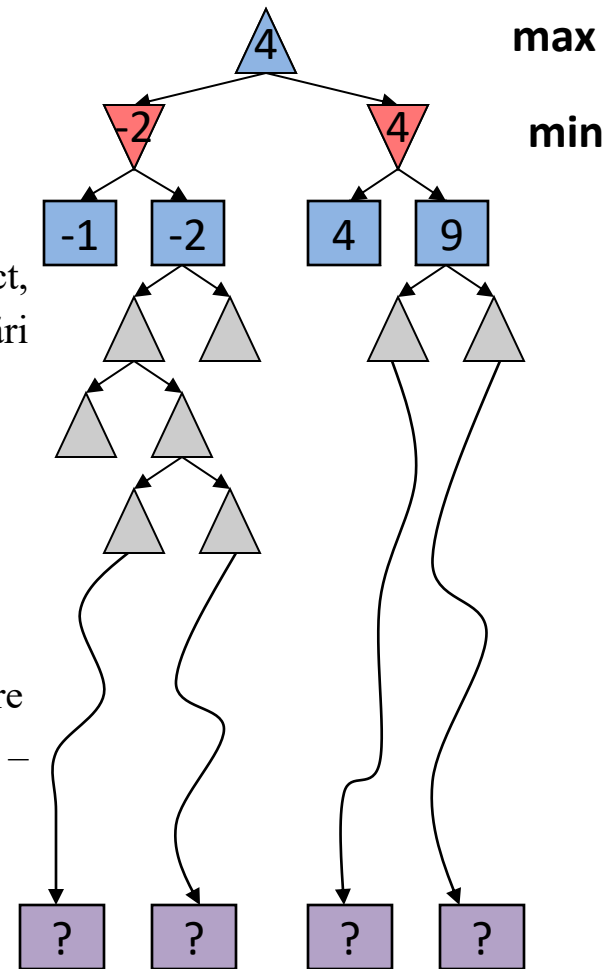
Limitări date de resurse

- problemă: în jocuri reale, nu putem genera tot arborele până la frunze din cauza resurselor limitate (timp + spațiu)

- soluție posibilă: căutare în adâncime limitată
 - caută până la o anumită adâncime în arborele de joc
 - înlocuiește utilitățile stărilor terminale (care sunt calculate exact, după regulile jocului) cu o funcție de evaluare a utilității unor stări neterminale (această funcție aproximează utilitatea)

- exemplu:
 - avem la dispoziție 100 de secunde pentru a realiza mutarea;
 - putem explora 10000 noduri / secundă;
 - în 100 de secunde putem explora 1 Milion de noduri pentru mutare
 - algoritmul α - β retezare (îl prezentăm azi) ajunge la adâncime 8 – performanță bună în șah;

- garanția de optimalitate dispare;
- cu cât pot analiza mai multe noduri cu atât mutarea este mai bună;
- uneori pot folosi și un algoritm de căutare iterative în adâncime



Implementare MiniMax (cu adâncime limitată d)

```
1. Function MaxValue(s, d)
2.   if (terminal(s) or d > maxD)
3.     return eval(s)
4.   v = -infinity
5.   for (c in children(s))
6.     v' = MinValue(c, d+1)
7.     if (v' > v) v = v'
8.   return v
```

```
1. Function MinValue(s, d)
2.   if (terminal(s) or d > maxD)
3.     return eval(s)
4.   v = +infinity
5.   for (c in children(s))
6.     v' = MaxValue(c, d+1)
7.     if (v' < v) v = v'
8.   return v
```

Implementare MiniMax (cu adâncime limitată d)

```
1.  Function MiniMax(s, d, max)
2.      if (terminal(s) or d > maxD)
3.          return eval(s)
4.      if (max)          // maximizing player
5.          v = -infinity
6.          for (c in children(s))
7.              v = max(v, MiniMax(c, d+1, false))
8.          return v
9.      else              // minimizing player
10.         v = +infinity
11.         for (c in children(s))
12.             v = min(v, MiniMax(c, d+1, true))
13.         return v
```

Apel inițial: *Minimax(start, 0, true)*

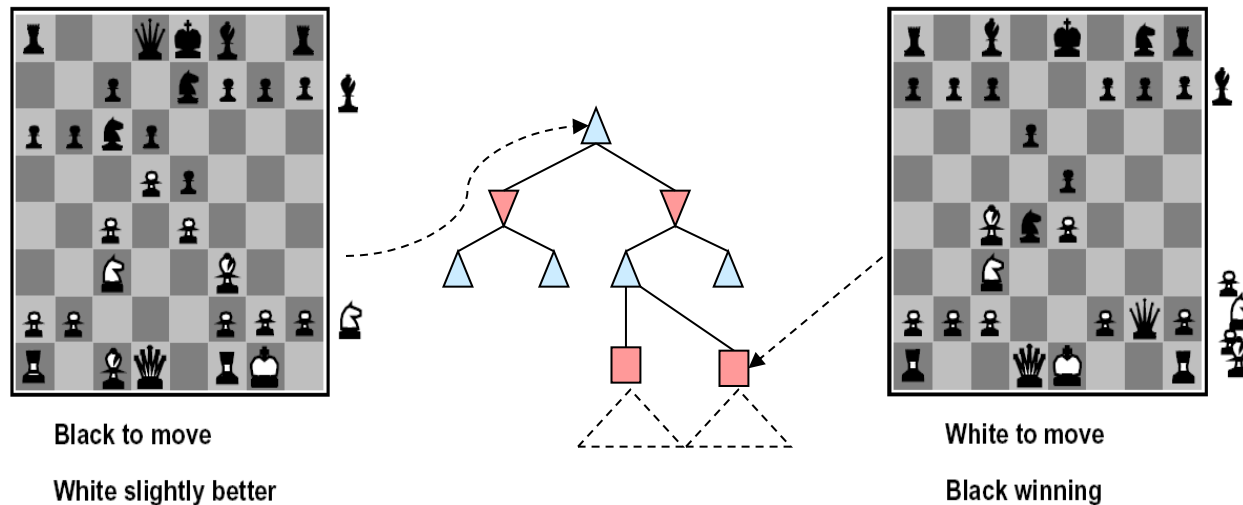
Influența adâncimii pentru funcția de evaluare

- Funcțiile de evaluare sunt imperfecte
- Cu cât merg mai adâncime în arbore și apelez funcția de evaluare a unei stări neterminale mai târziu cu atât am șanse să greșesc mai puțin.



Funcții de evaluare - șah

- Funcțiile de evaluare asociază un scor stărilor neterminale (este folosită de căutarea în adâncime limitată, căutarea iterativă în adâncime)

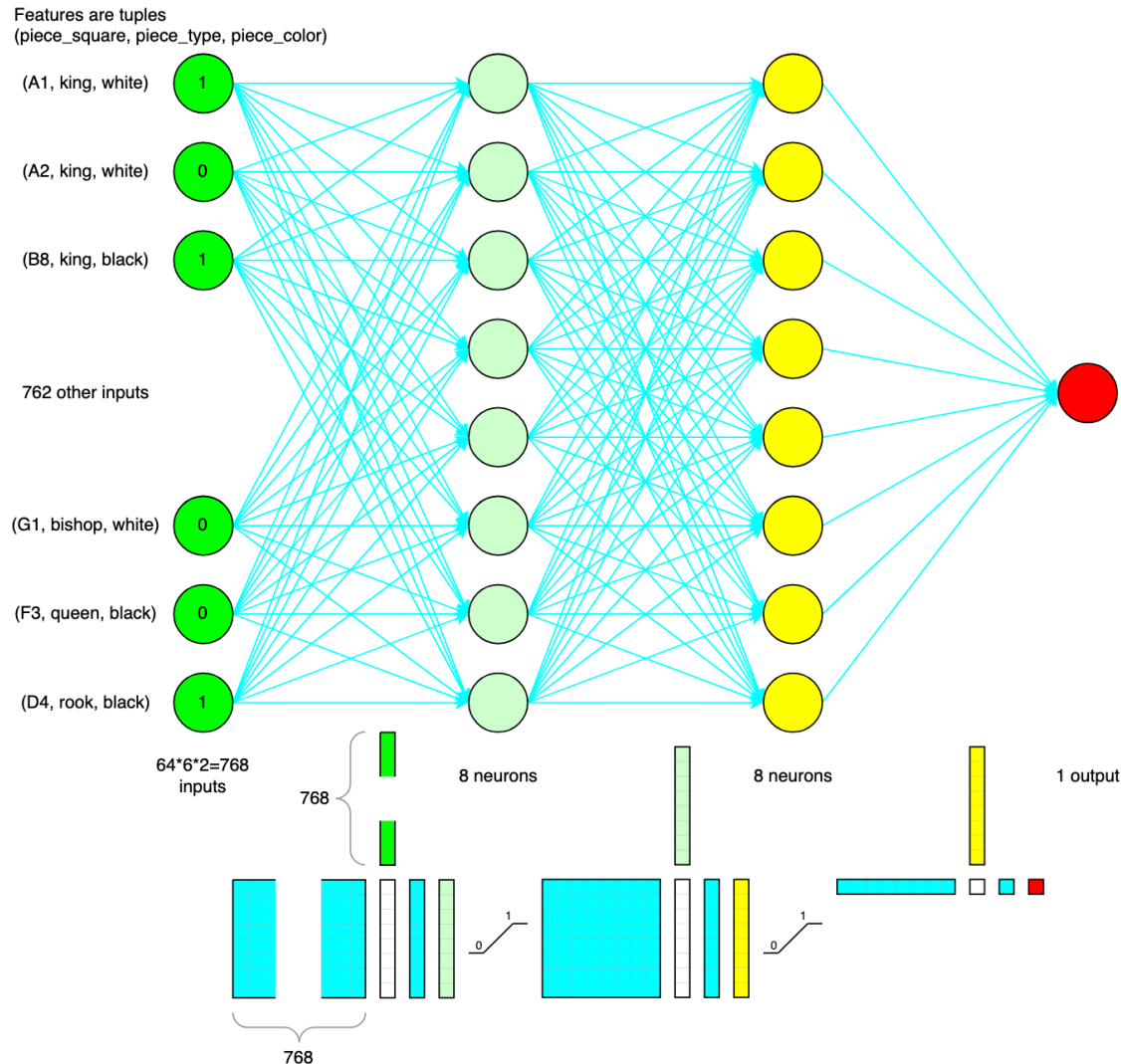


- o funcție de evaluare ideală returnează valoarea MiniMax a stării.
- diverse funcții de evaluare specifice pentru fiecare joc: pentru șah se consideră funcții liniare în care ponderăm piesele de pe tablă

$$Eval(s) = w_1 f_1(s) + w_2 f_2(s) + \dots + w_n f_n(s)$$

- unde, $f_1(s) = (\text{\#regine_albe} - \text{\#regine_negre})$, $w_1 = 100$, etc.

Funcții de evaluare - StockFISH

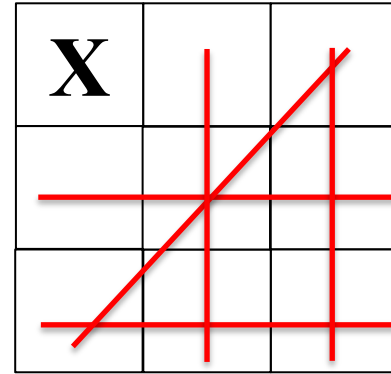
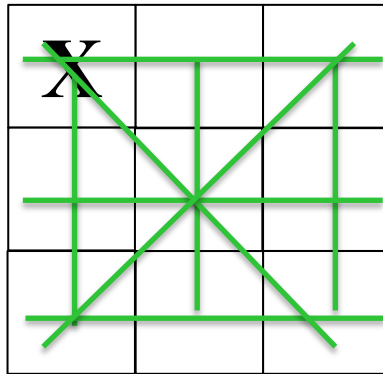
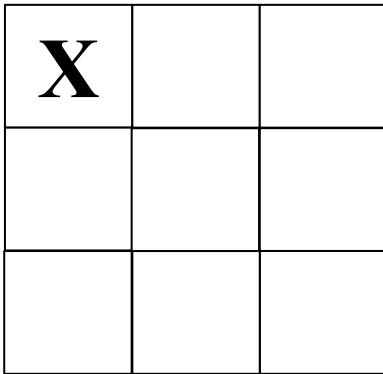


NNUE - Efficiently Updatable Neural Network pentru a cuantifica scorul unei poziții

<https://official-stockfish.github.io/docs/nnue-pytorch-wiki/docs/nnue.html#what-is-nnue>

Funcții de evaluare – X și 0

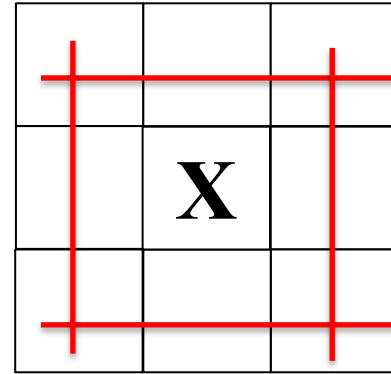
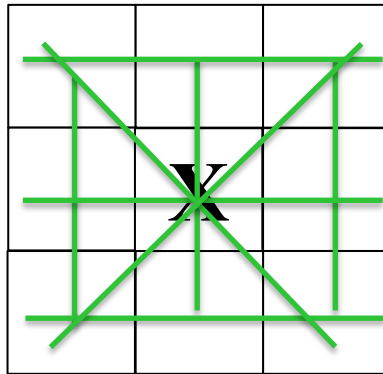
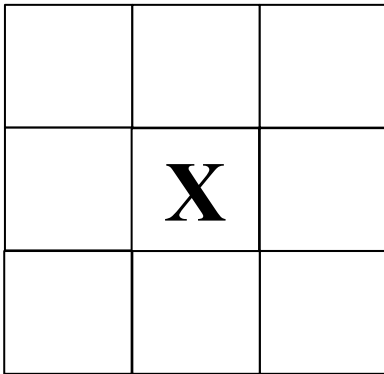
- o funcție de evaluare ideală returnează valoarea MiniMax a stării.
- diverse funcții de evaluare specifice pentru fiecare joc:



- exemplu de funcție de evaluare pentru X și 0: diferența dintre numărul de soluții disponibile pentru MAX și numărul de soluții disponibile pentru MIN
- în exemplul de mai sus: $8 - 5 = 3$

Funcții de evaluare – X și 0

- o funcție de evaluare ideală returnează valoarea MiniMax a stării.
- diverse funcții de evaluare specifice pentru fiecare joc:



- exemplu de funcție de evaluare pentru X și 0: diferența dintre numărul de soluții disponibile pentru MAX și numărul de soluții disponibile pentru MIN
- În exemplul de mai sus: $8 - 4 = 4$

Funcții de evaluare – X și 0

- o funcție de evaluare ideală returnează valoarea MiniMax a stării.
- diverse funcții de evaluare specifice pentru fiecare joc:

	X	O

	X	O

	X	O

- exemplu de funcție de evaluare pentru X și 0: diferența dintre numărul de soluții disponibile pentru MAX și numărul de soluții disponibile pentru MIN
- În exemplul de mai sus: $6 - 4 = 2$

Funcții de evaluare – X și 0

- o funcție de evaluare ideală returnează valoarea MiniMax a stării.
- diverse funcții de evaluare specifice pentru fiecare joc:

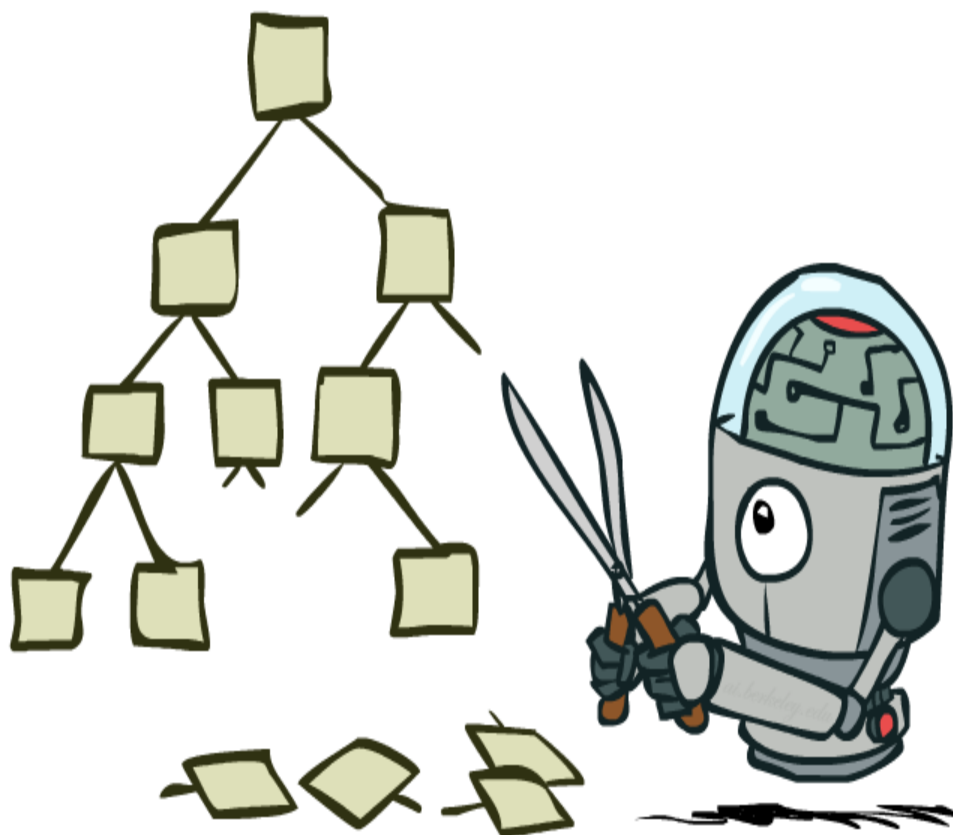
		0
	X	

		0
	X	

		0
	X	

- exemplu de funcție de evaluare pentru X și 0: diferența dintre numărul de soluții disponibile pentru MAX și numărul de soluții disponibile pentru MIN
- În exemplul de mai sus: $5 - 4 = 1$

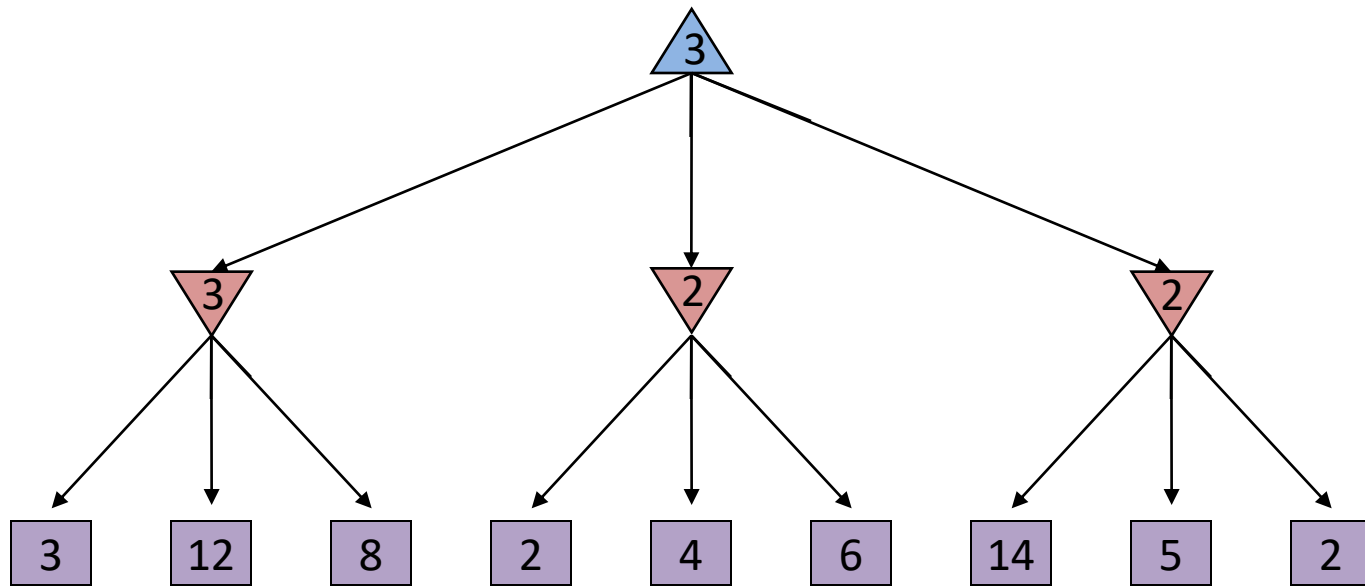
Retezări (Pruning) în arborele de joc



Exemplu MiniMax

MAX

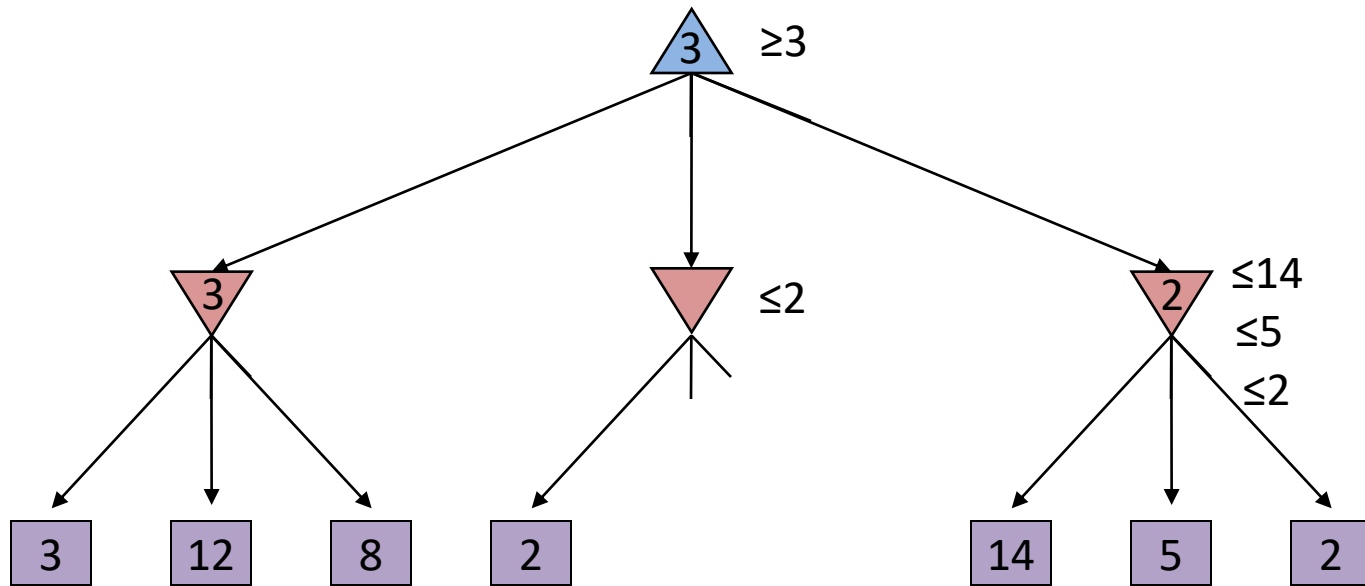
MIN



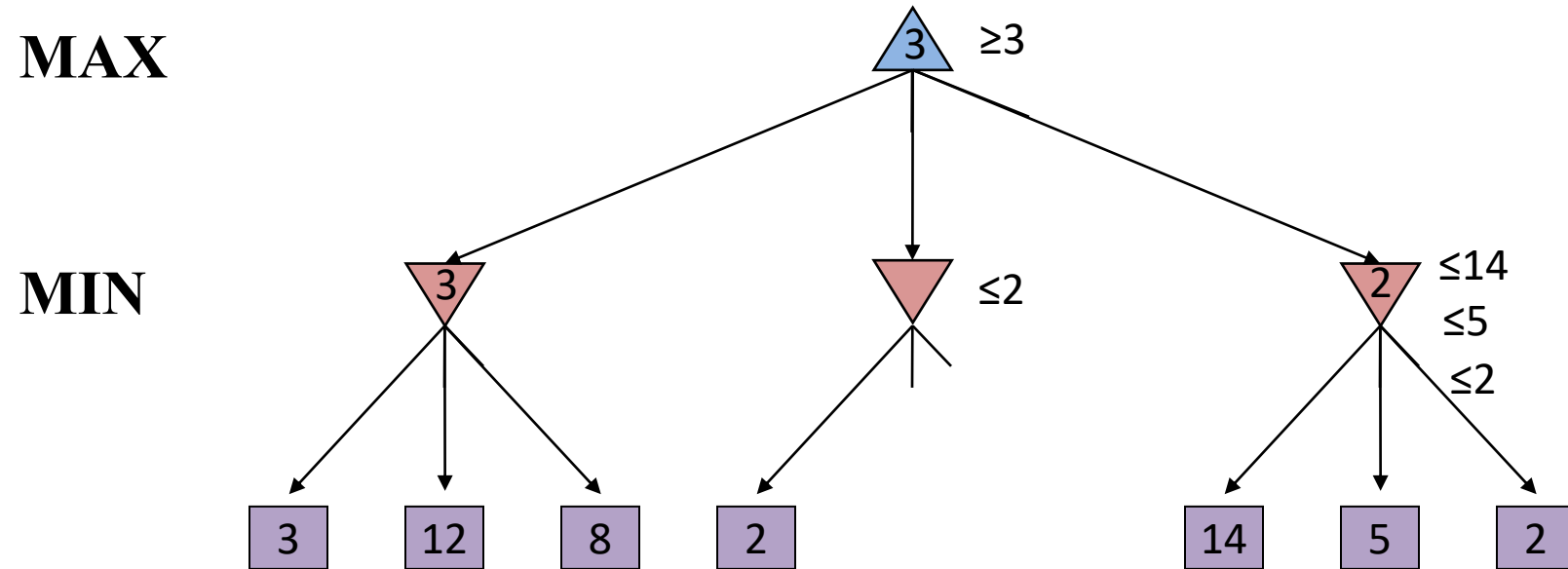
Accelerarea algoritmului MiniMax

MAX

MIN



Retezare alfa-beta



Tehnica de alfa-beta retezare, când este aplicată unui arbore de tip Minimax standard, **va întoarce aceeași mutare pe care ar furniza-o și Algoritmul MiniMax, dar într-un timp mai scurt**, întrucât realizează o retezare a unor ramuri (subarbori) ale arborelui care nu pot influența decizia finală și care nu mai sunt vizitate.

Retezare alfa-beta



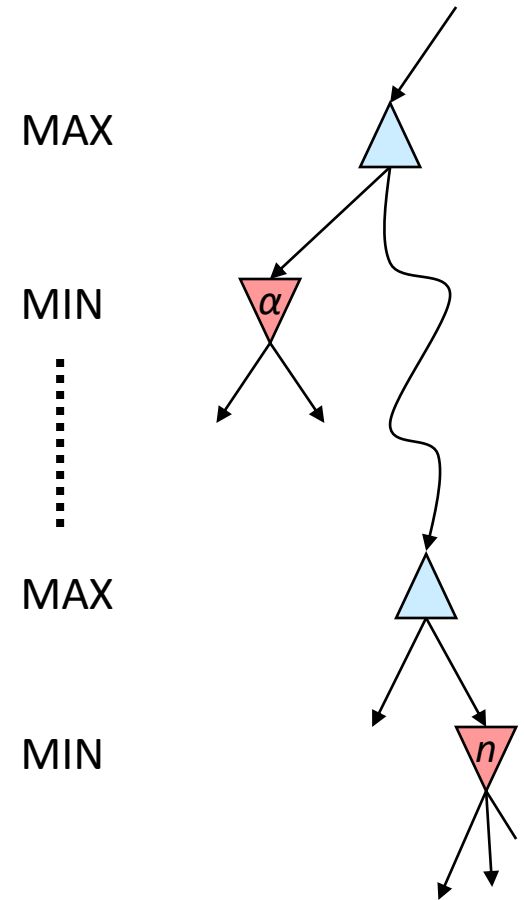
înainte



după

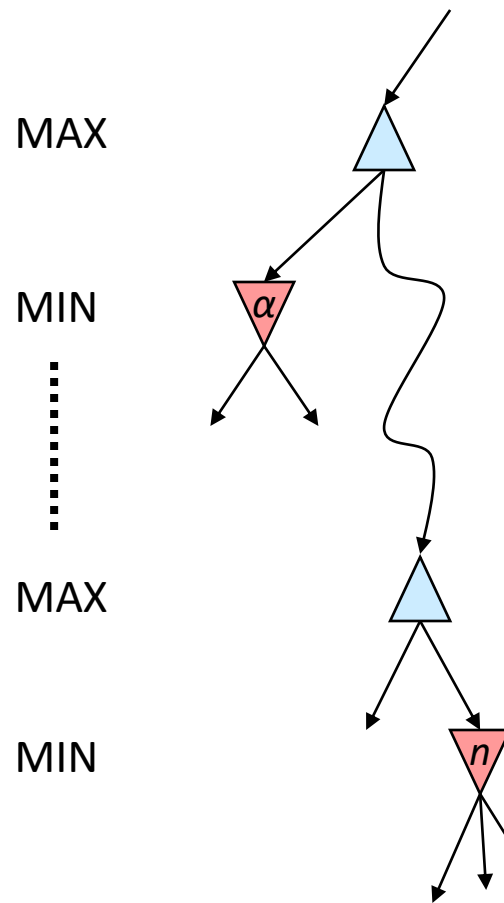
Retezare alfa-beta (alfa-beta pruning)

- Cazurile când se aplică (pentru noduri de tip MIN)
 - calculăm pentru jucătorul MIN valoarea nodului n
 - iterăm după toți succesorii nodului n de la stânga la dreapta
 - valoarea estimată a nodului n va scădea pe măsură ce îi vizităm toți succesorii
 - cine este interesat de valoarea nodului n ? MAX
 - fie α cea mai bună valoare curentă (garantată) pe care MAX o poate obține după ce a vizitat subarborii din stânga subarborelui actual vizitat pornind de la rădăcină
 - dacă valoarea curentă estimată a lui n devine mai mică decât α , MAX nu va alege acest drum, deci putem să renunțăm (să retezăm) să evaluăm valorile pentru celelalte noduri succesori ale lui n .
- Cazurile când se aplică (pentru noduri de tip MAX)
 - simetric, se definește în mod similar valoarea β ca fiind cea mai bună valoare (garantată) pe care MIN o poate obține.
 - MIN nu mai trebuie să mai ia în considerare nicio valoare internă mai mare sau egală cu β care este asociată oricărui nod intern n de tip MAX. Putem tăia (reteza) acea ramură a arborelui de joc.



Retezare alfa-beta (alfa-beta pruning)

- Încep cu parametri $\alpha = -\infty$, $\beta = +\infty$ din rădăcină pe care îi transmit (*ca valoare nu ca referință*) în subarbori și după ce îi parcurg pe aceștia actualizez valorile α și β .
- Pentru nodurile de tip MAX: după ce parcurg prima ramură din stânga (sub-arborele stâng) am o valoare minimă α . Pot atunci pentru sub-arborii care urmează să nu mai vizitez (să retez) sub-arbori din nodurile de tip MIN pentru care avem $\beta \leq \alpha$.
- Pentru nodurile de tip MIN: după ce parcurg prima ramură din stânga (sub-arborele stâng) am o valoare maximă β . Pot atunci pentru sub-arborii care urmează să nu mai vizitez (să retez) sub-arbori din nodurile de tip MAX pentru care avem $\alpha \geq \beta$.



Implementare Alfa-Beta retezare

α : opțiunea curentă cea mai bună a lui MAX (MAX vrea să maximizeze α)
de la nodul curent la nodul rădăcină

β : opțiunea curentă cea mai bună a lui MIN (MIN vrea să minimizeze β) de
la nodul curent la nodul rădăcină

Întotdeauna vom avea $\alpha \leq \beta$

```
def max-value(stare,  $\alpha$ ,  $\beta$ ): //nodul  $n$  e de tip MAX, MAX are  $\alpha$ , MIN are  $\beta$ 
    initializeaza  $v = -\infty$ 
    pentru fiecare succesor al stării: //pentru fiecare succesor al lui  $n$ 
         $v = \max(v, \text{value}(\text{successor}, \alpha, \beta))$  //calculăm valoarea  $v$  și actualizăm max
        if  $v \geq \beta$  return  $v$  //dacă  $v$  depășește limita  $\beta$  a lui MIN nu mai continuăm
        // întoarcem în acest caz  $v =$  o estimare a lui  $n$ 
         $\alpha = \max(\alpha, v)$  // actualizăm valoarea  $\alpha$ 
    return  $v$ 
```

Implementare Alfa-Beta retezare

α : opțiunea curentă cea mai bună a lui MAX (MAX vrea să maximizeze α) de la nodul curent la nodul rădăcină

β : opțiunea curentă cea mai bună a lui MIN (MIN vrea să minimizeze β) de la nodul curent la nodul rădăcină

Întotdeauna vom avea $\alpha \leq \beta$

def max-value(stare, α , β):

 initializeaza $v = -\infty$

 pentru fiecare succesor al stării:

$v = \max(v, \text{value}(\text{successor}, \alpha, \beta))$

 if $v \geq \beta$ returneaza v

$\alpha = \max(\alpha, v)$

 returneaza v

def min-value(stare , α , β):

 initializeaza $v = +\infty$

 pentru fiecare succesor al stării:

$v = \min(v, \text{value}(\text{successor}, \alpha, \beta))$

 if $v \leq \alpha$ returneaza v

$\beta = \min(\beta, v)$

 returneaza v

Implementare MiniMax

def value(stare):

dacă stare este o stare terminală: returnează utilitatea stării

dacă următorul agent care mută este **MAX**:
returnează **max-value(stare)**

dacă următorul agent care mută este **MIN**: returnează **min-value(stare)**

def max-value(stare):

initializeaza $v = -\infty$

pentru fiecare successor al stării:

$v = \max(v, \text{value}(\text{successor}))$

returneaza v

def min-value(stare):

initializeaza $v = +\infty$

pentru fiecare succesor al stării:

$v = \min(v, \text{value}(\text{successor}))$

returneaza v

$$V(s) = \max_{s' \in \text{Successors}(s)} V(s')$$

$$V(s) = \min_{s' \in \text{Successors}(s)} V(s')$$

Implementare Alfa-Beta retezare

def value(stare, α , β):

 dacă stare este o stare terminală: returnează utilitatea stării

 dacă următorul agent care mută este MAX:

 returnează max-value(stare, α , β)

 dacă următorul agent care mută este MIN: returnează min-value(stare, α , β)

def max-value(stare, α , β):

 initializează $v = -\infty$

 pentru fiecare succesor al stării:

$v = \max(v, \text{value}(\text{successor}, \alpha, \beta))$

 if $v \geq \beta$ returnează v

$\alpha = \max(\alpha, v)$

 returnează v

def min-value(stare, α , β):

 initializează $v = +\infty$

 pentru fiecare succesor al stării:

$v = \min(v, \text{value}(\text{successor}, \alpha, \beta))$

 if $v \leq \alpha$ returnează v

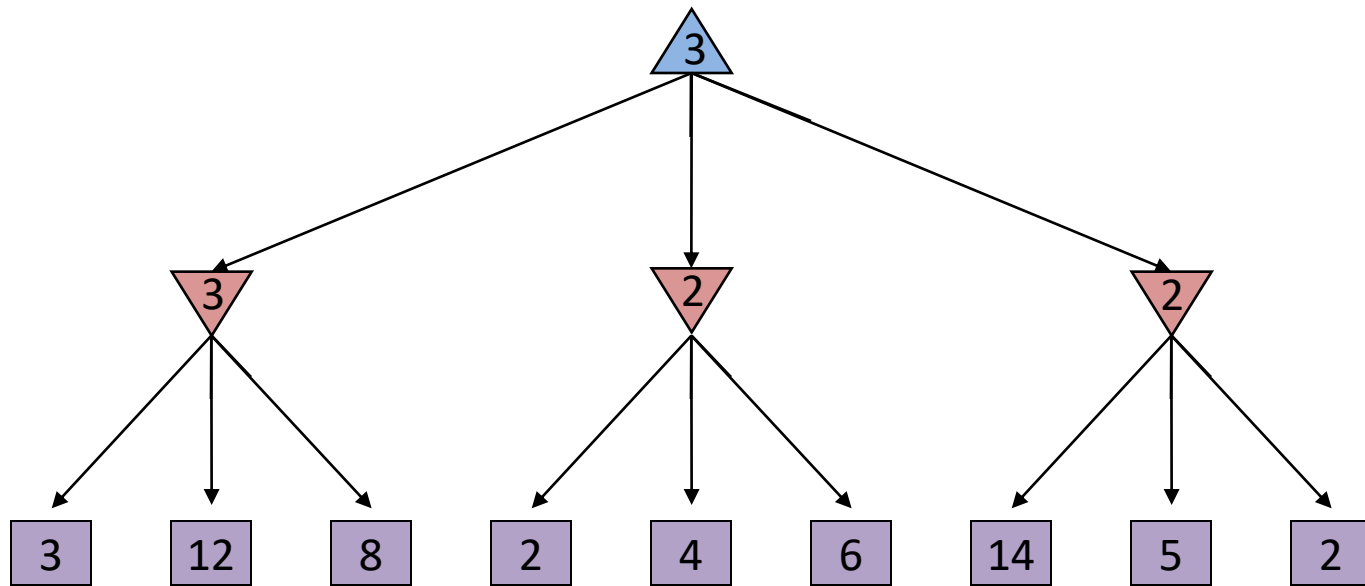
$\beta = \min(\beta, v)$

 returnează v

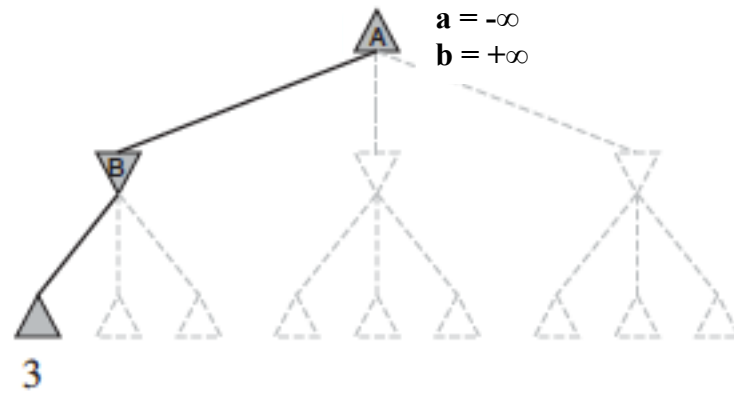
Exemplu MiniMax

MAX

MIN



(a)



def max-value(stare, α , β):

 initializeaza $v = -\infty$

 pentru fiecare succesor al stării:

$v = \max(v, \text{value}(\text{successor}, \alpha, \beta))$

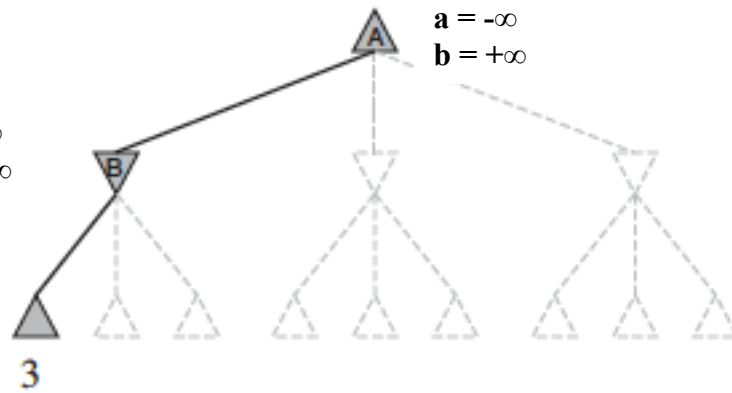
 if $v \geq \beta$ returneaza v

$\alpha = \max(\alpha, v)$

 returneaza v

(a)

$a = -\infty$
 $b = +\infty$



def min-value(stare , α , β):

 initializeaza $v = +\infty$

 pentru fiecare succesor al stării:

$v = \min(v, \text{value}(\text{successor}, \alpha, \beta))$

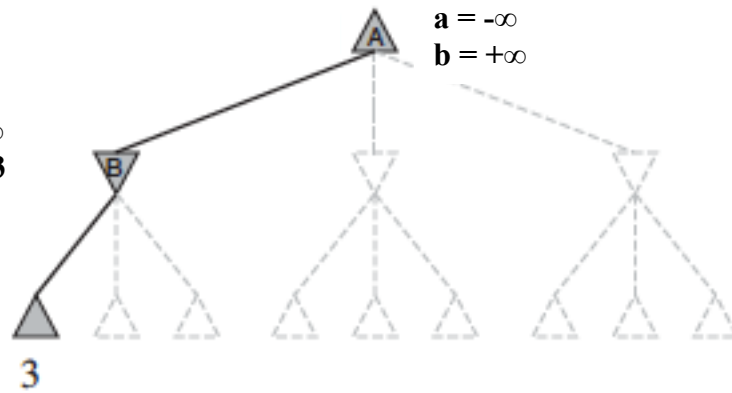
 if $v \leq \alpha$ returneaza v

$\beta = \min(\beta, v)$

 returneaza v

(a)

$a = -\infty$
 $b = +3$



def min-value(stare , α , β):

 initializeaza $v = +\infty$

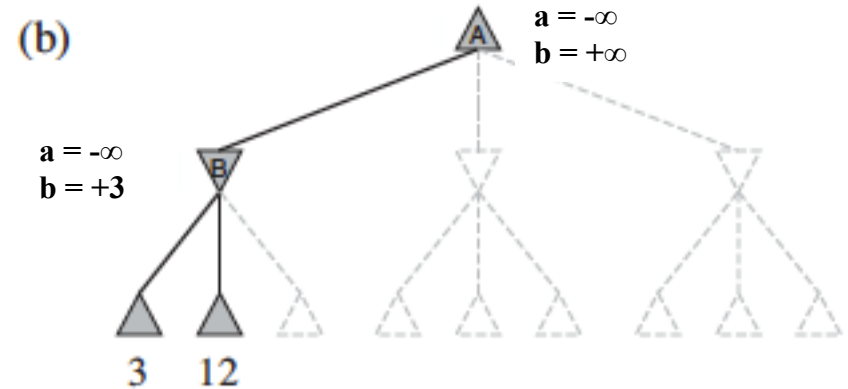
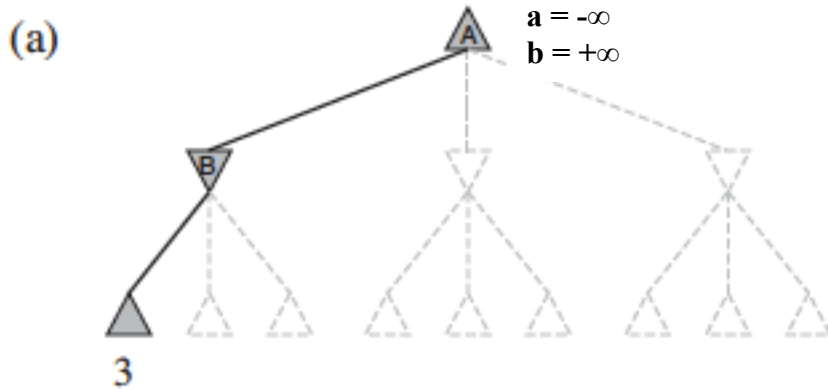
 pentru fiecare succesor al stării:

$v = \min(v, \text{value}(\text{successor}, \alpha, \beta))$

 if $v \leq \alpha$ returneaza v

$\beta = \min(\beta, v)$

 returneaza v



def min-value(stare , α , β):

 initializeaza $v = +\infty$

 pentru fiecare succesori al stării:

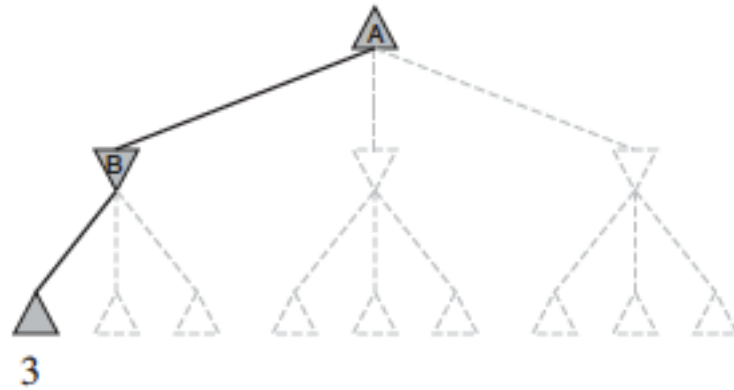
$v = \min(v, \text{value}(\text{successor}, \alpha, \beta))$

 if $v \leq \alpha$ returneaza v

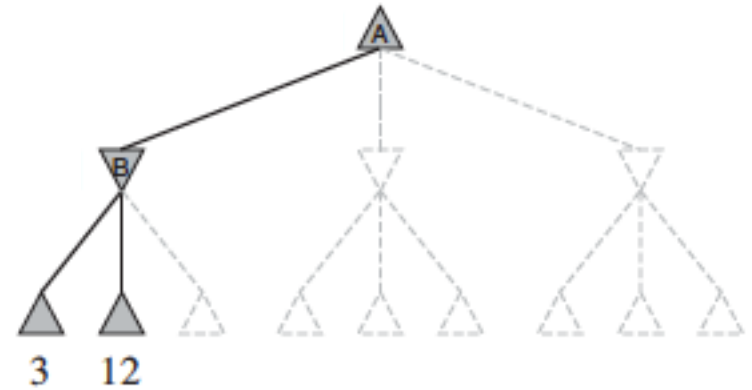
$\beta = \min(\beta, v)$

 returneaza v

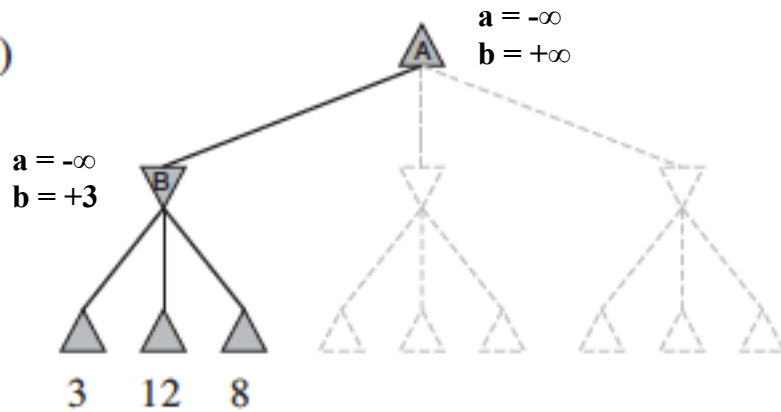
(a)



(b)



(c)



def min-value(stare , α , β):

 initializeaza $v = +\infty$

 pentru fiecare succesor al stării:

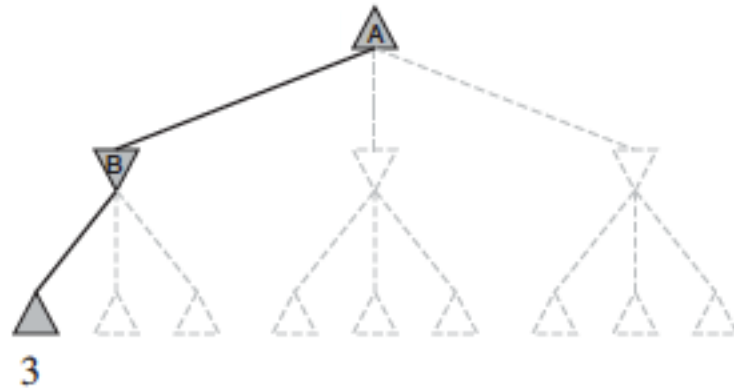
$v = \min(v, \text{value}(\text{successor}, \alpha, \beta))$

 if $v \leq \alpha$ returneaza v

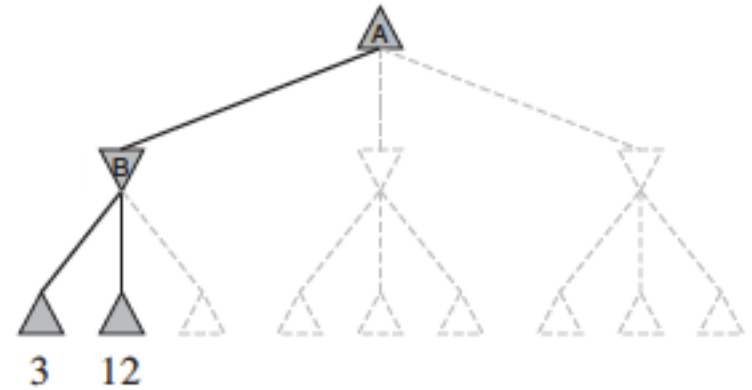
$\beta = \min(\beta, v)$

 returneaza v

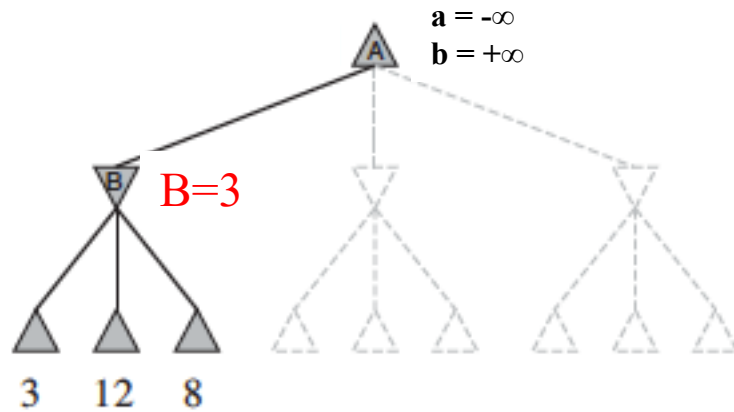
(a)



(b)



(c)



def max-value(stare, α , β):

 initializeaza $v = -\infty$

 pentru fiecare succesor al stării:

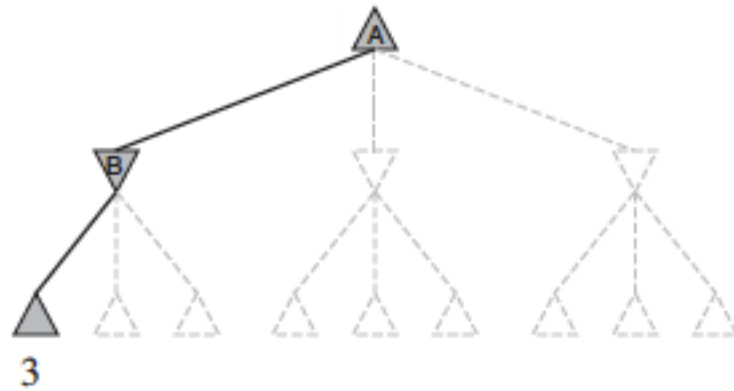
$v = \max(v, \text{value}(\text{successor}, \alpha, \beta))$

 if $v \geq \beta$ returneaza v

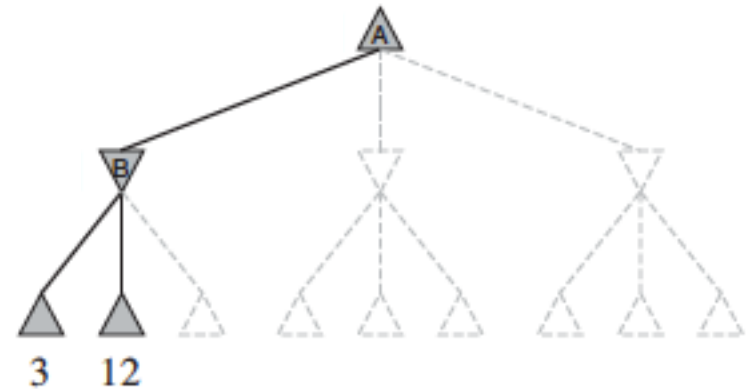
$\alpha = \max(\alpha, v)$

 returneaza v

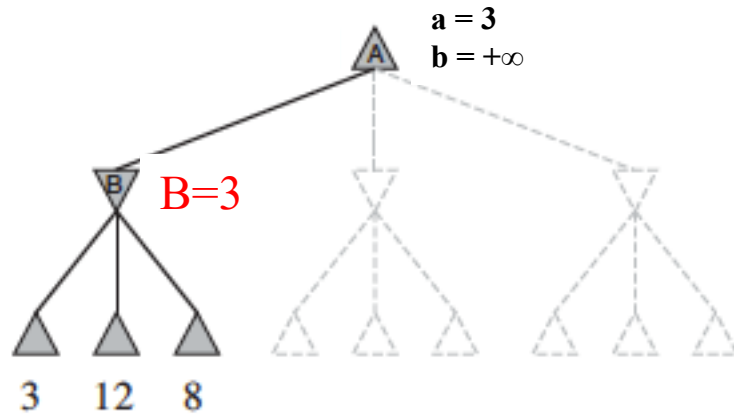
(a)



(b)



(c)



def max-value(stare, α , β):

 initializeaza $v = -\infty$

 pentru fiecare succesor al stării:

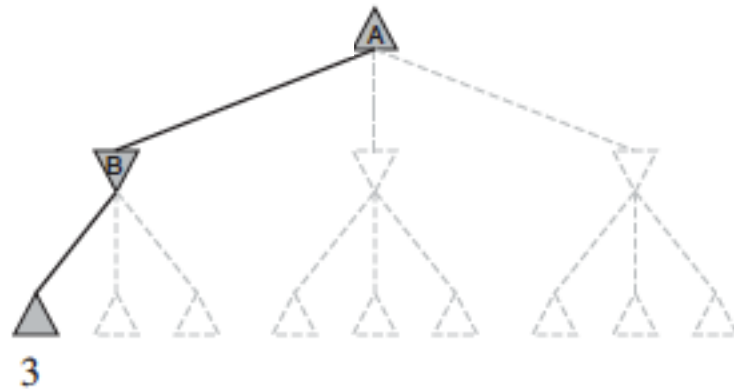
$v = \max(v, \text{value}(\text{successor}, \alpha, \beta))$

 if $v \geq \beta$ returneaza v

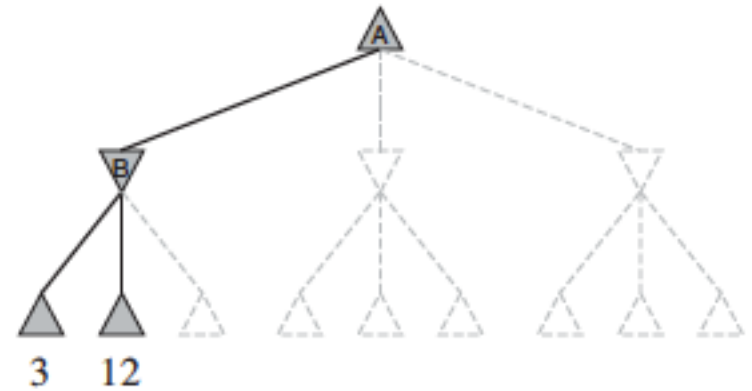
$\alpha = \max(\alpha, v)$

 returneaza v

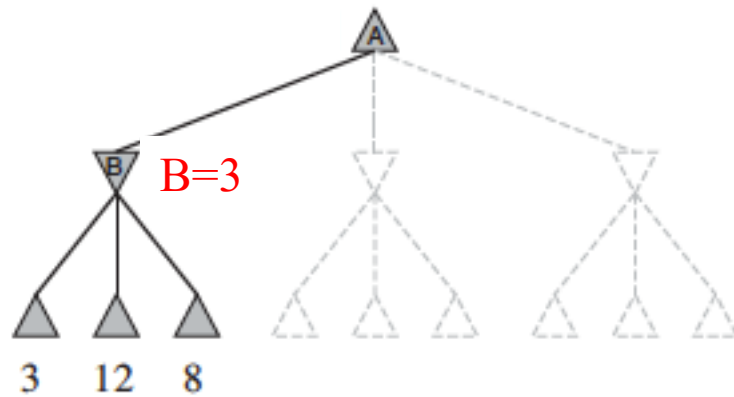
(a)



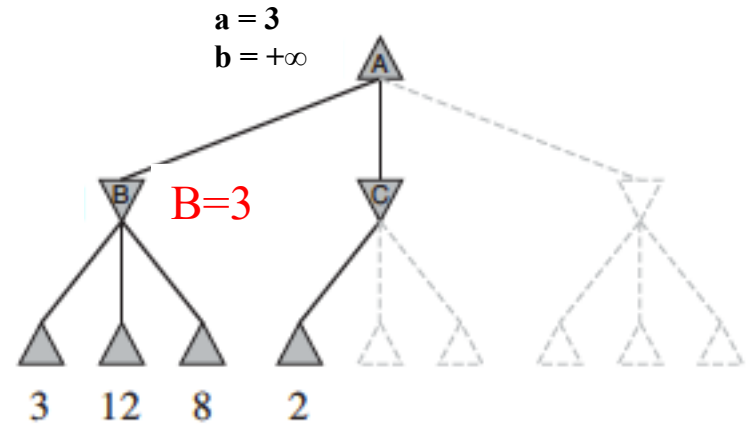
(b)



(c)



(d)



def min-value(stare , α , β):

 initializeaza $v = +\infty$

 pentru fiecare succesor al stării:

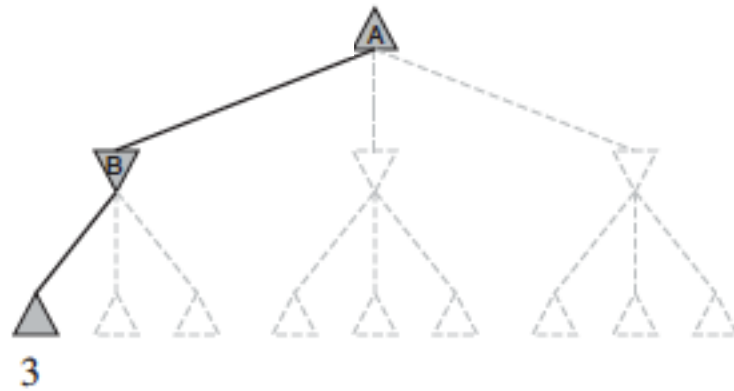
$v = \min(v, \text{value}(\text{successor}, \alpha, \beta))$

 if $v \leq \alpha$ returneaza v

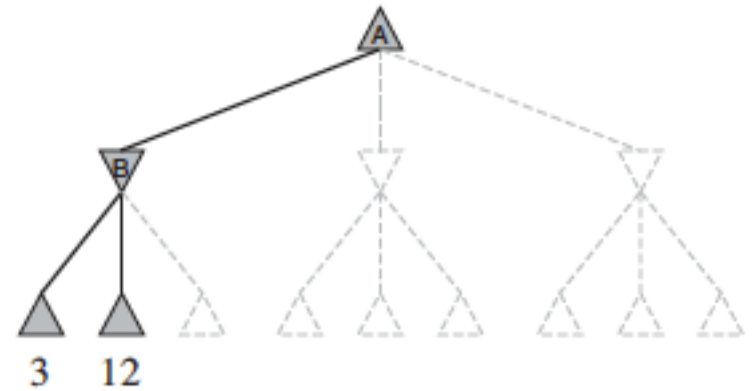
$\beta = \min(\beta, v)$

 returneaza v

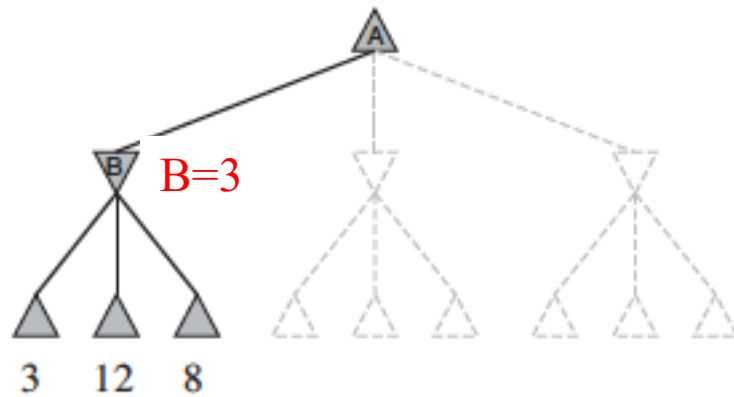
(a)



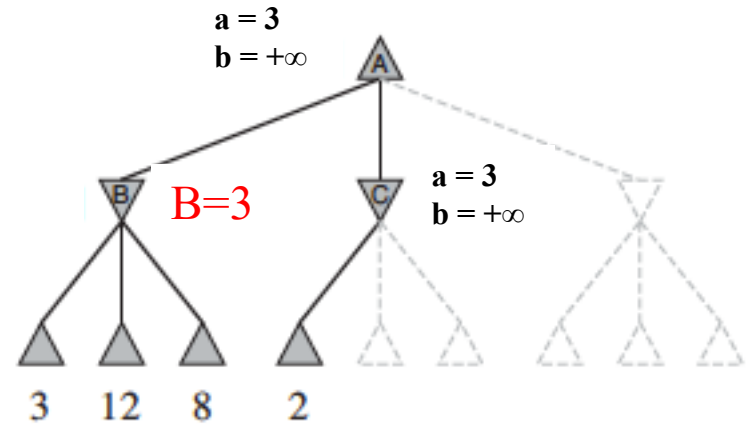
(b)



(c)



(d)



def min-value(stare , α , β):

 initializeaza $v = +\infty$

 pentru fiecare succesor al stării:

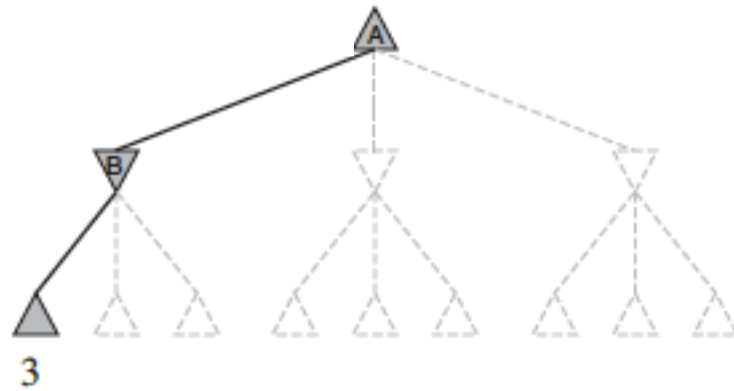
$v = \min(v, \text{value}(\text{successor}, \alpha, \beta))$

 if $v \leq \alpha$ returneaza v

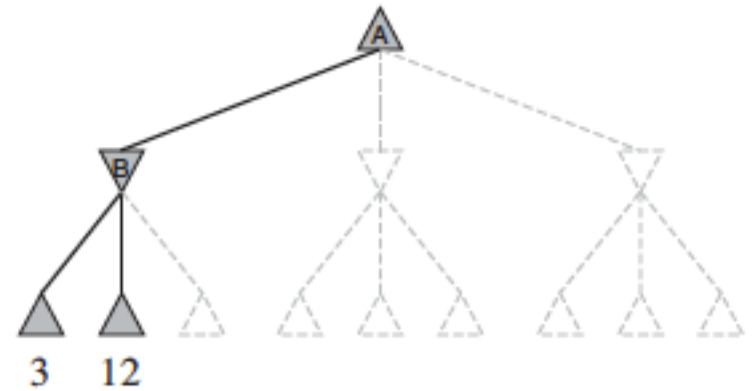
$\beta = \min(\beta, v)$

 returneaza v

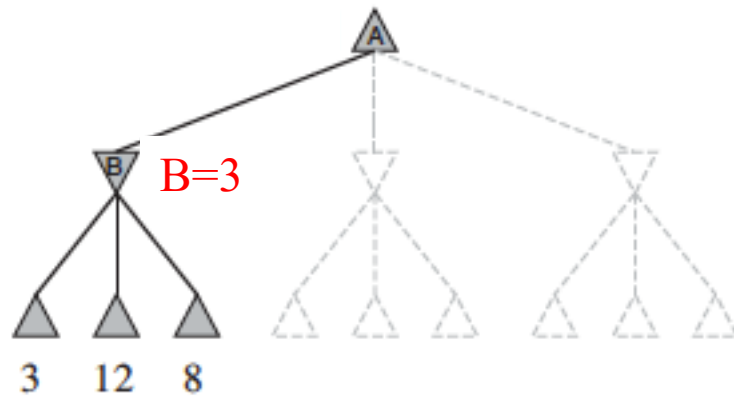
(a)



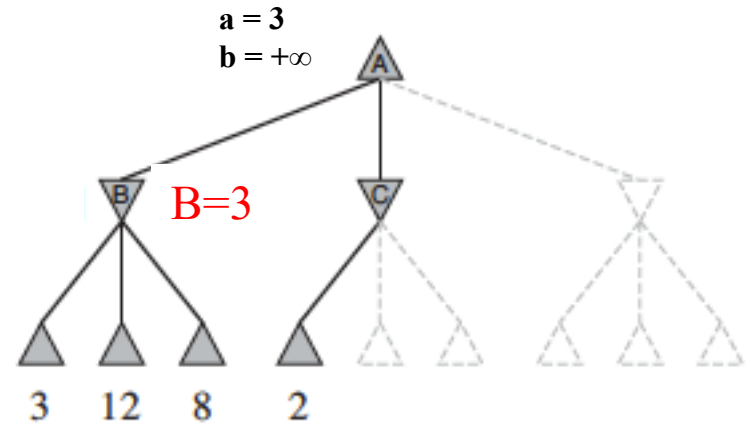
(b)



(c)



(d)



$a = 3$
 $b = +\infty$

$a = 3$
 $v = 2$

def min-value(stare , α , β):

 initializeaza $v = +\infty$

 pentru fiecare succesori al stării:

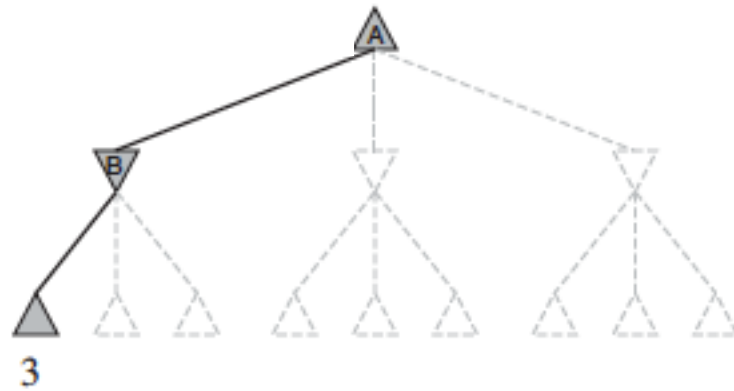
$v = \min(v, \text{value}(\text{successor}, \alpha, \beta))$

 if $v \leq \alpha$ returneaza v

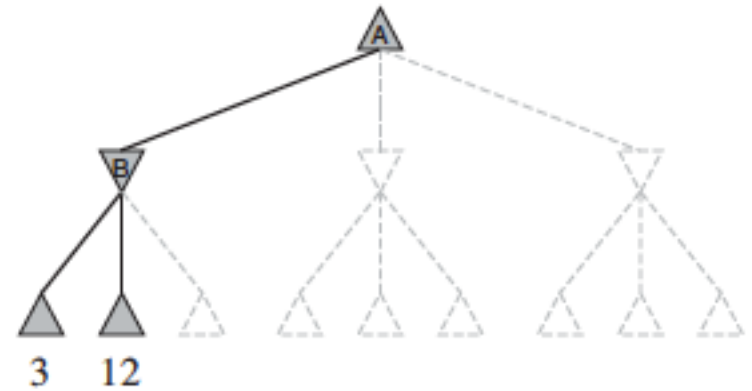
$\beta = \min(\beta, v)$

 returneaza v

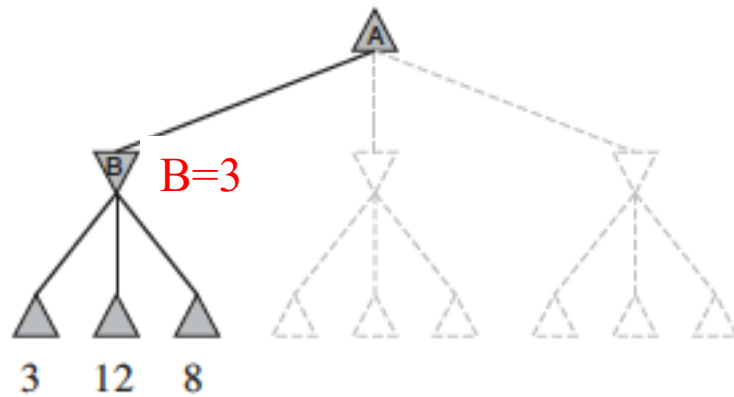
(a)



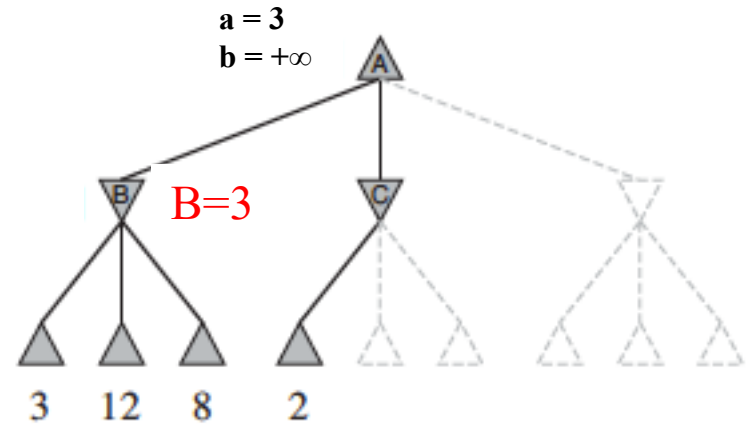
(b)



(c)



(d)



def min-value(stare , α , β):

 initializeaza $v = +\infty$

 pentru fiecare succesori al stării:

$v = \min(v, \text{value}(\text{successor}, \alpha, \beta))$

 if $v \leq \alpha$ returneaza v

$\beta = \min(\beta, v)$

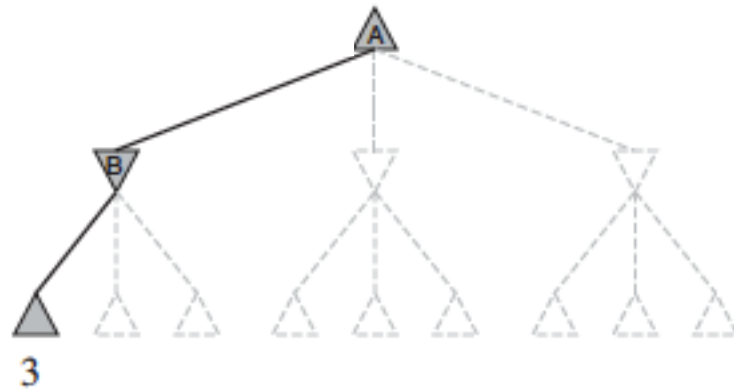
 returneaza v

$a = 3$

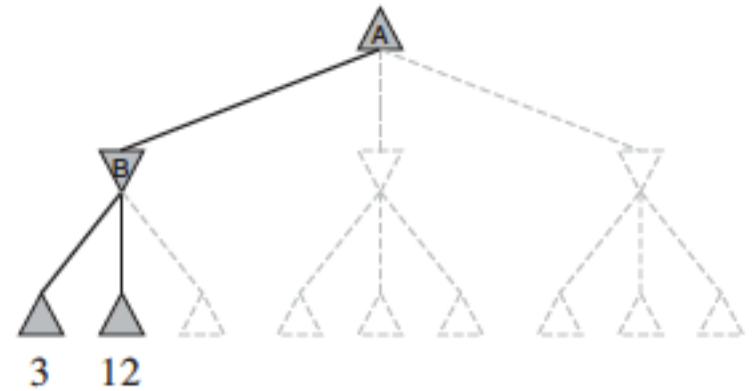
$v = 2$

pruning

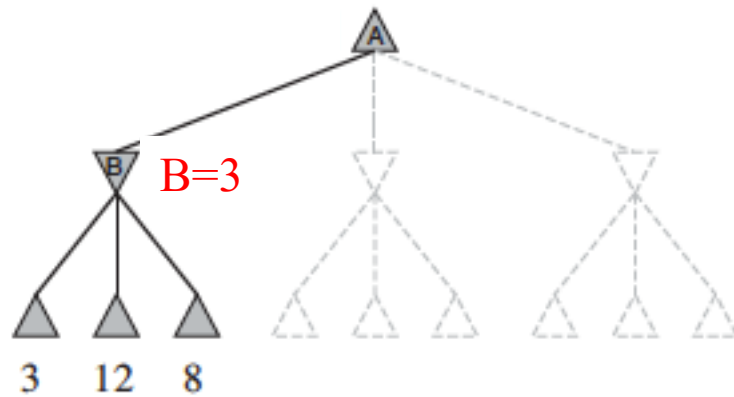
(a)



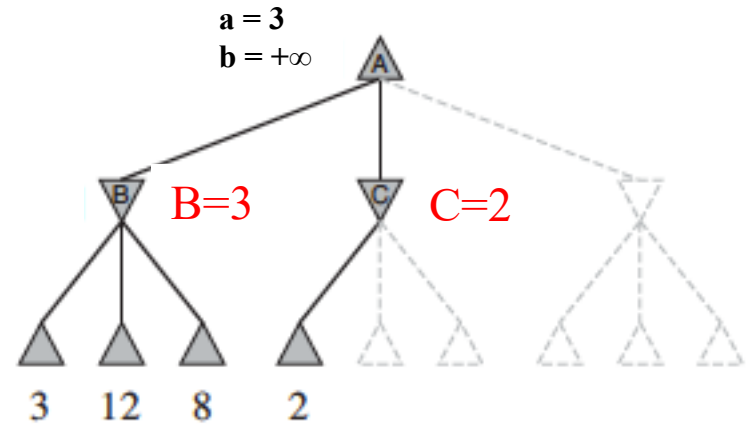
(b)



(c)



(d)



def min-value(stare , α , β):

 initializeaza $v = +\infty$

 pentru fiecare succesori al stării:

$v = \min(v, \text{value}(\text{successor}, \alpha, \beta))$

 if $v \leq \alpha$ returneaza v

$\beta = \min(\beta, v)$

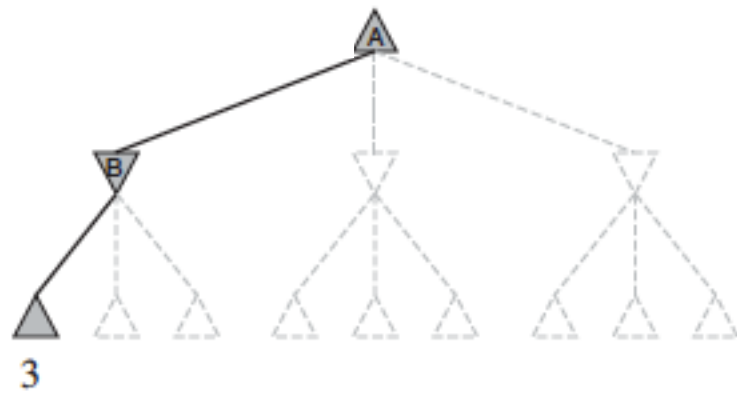
 returneaza v

$a = 3$

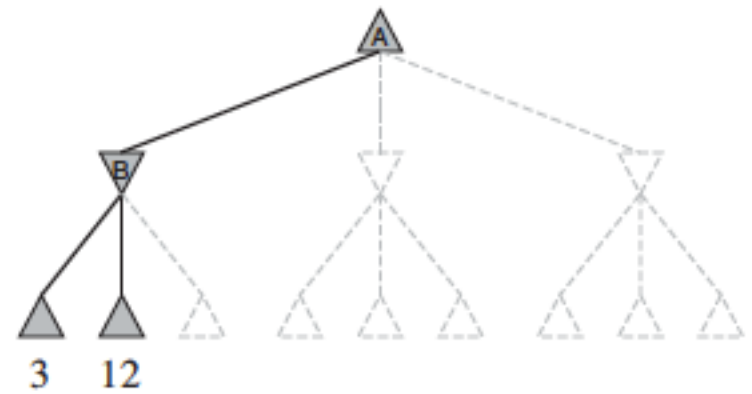
$v = 2$

pruning

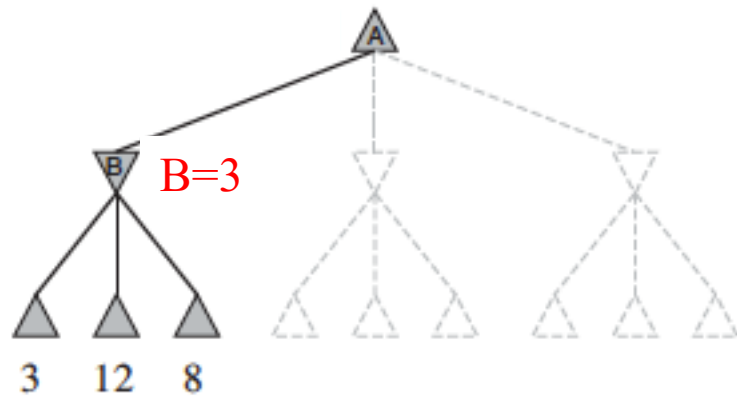
(a)



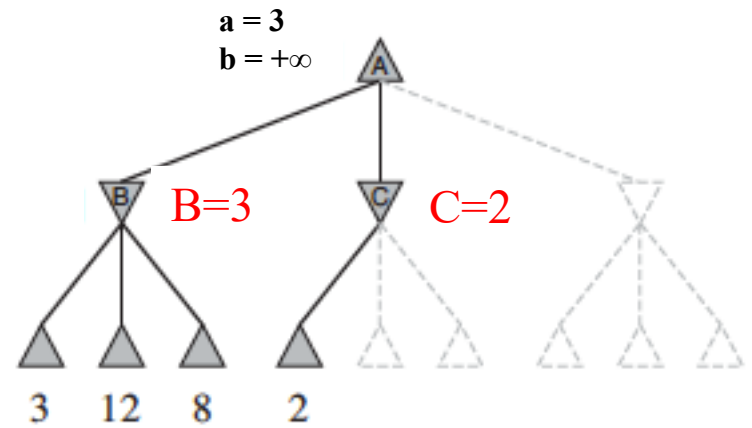
(b)



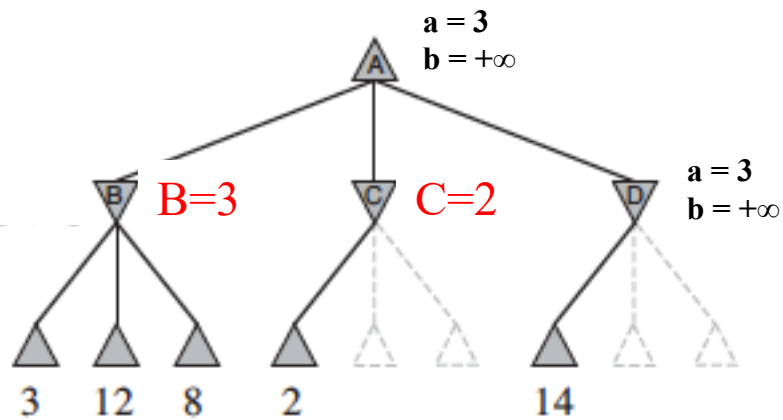
(c)



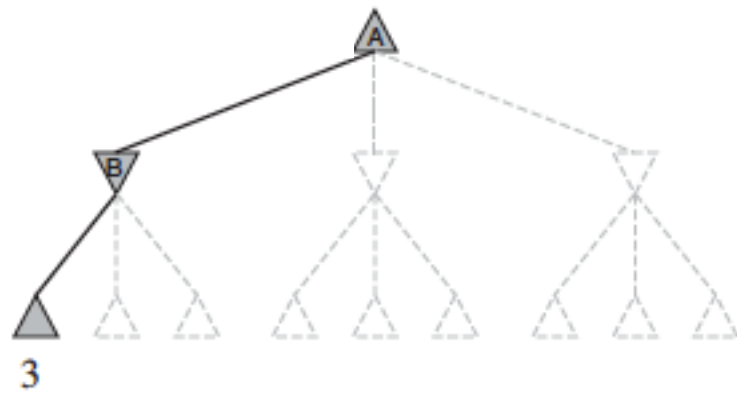
(d)



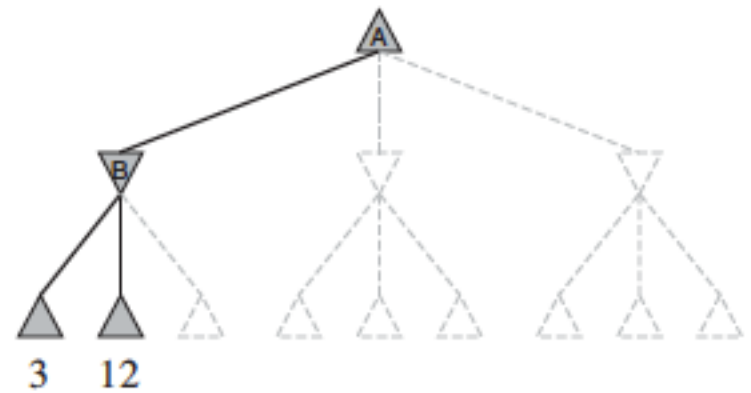
(e)



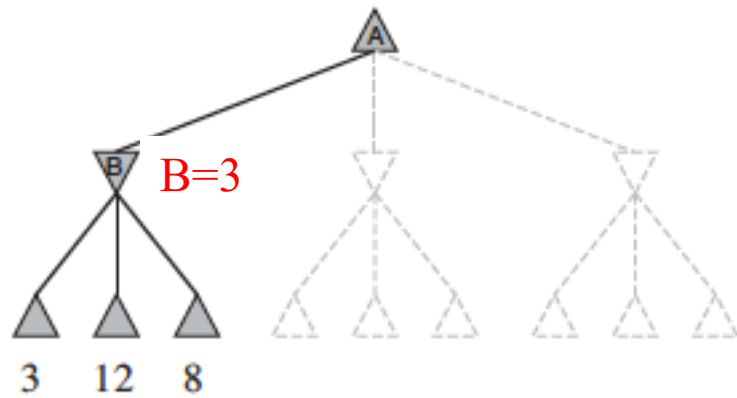
(a)



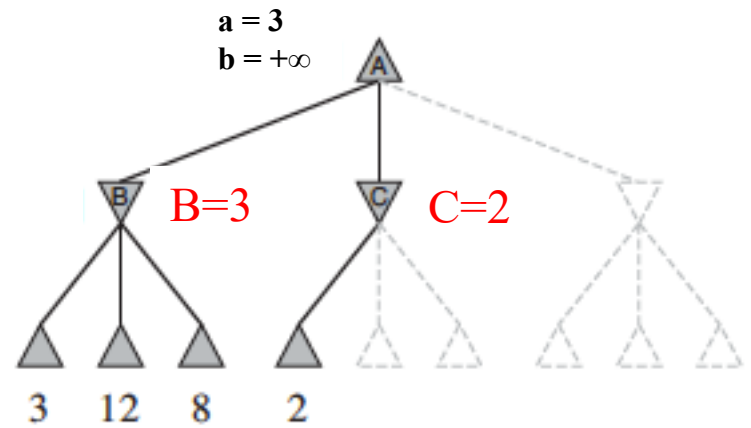
(b)



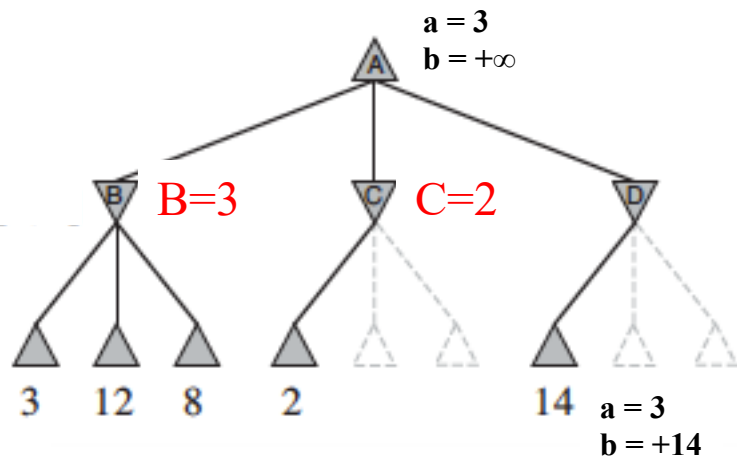
(c)



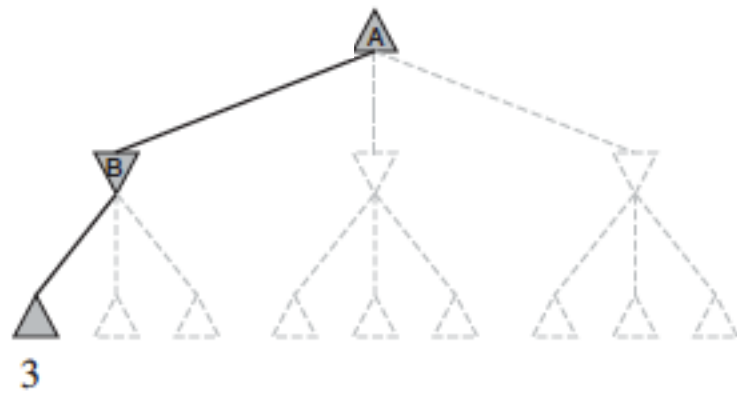
(d)



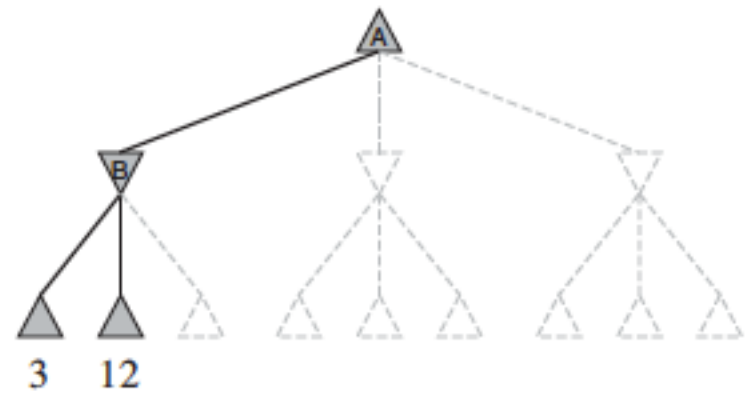
(e)



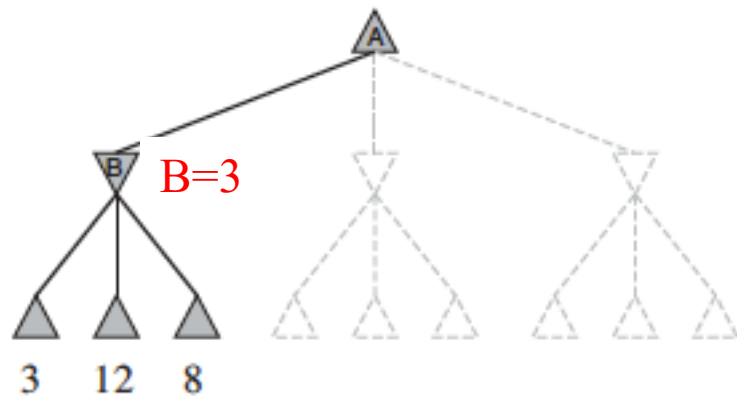
(a)



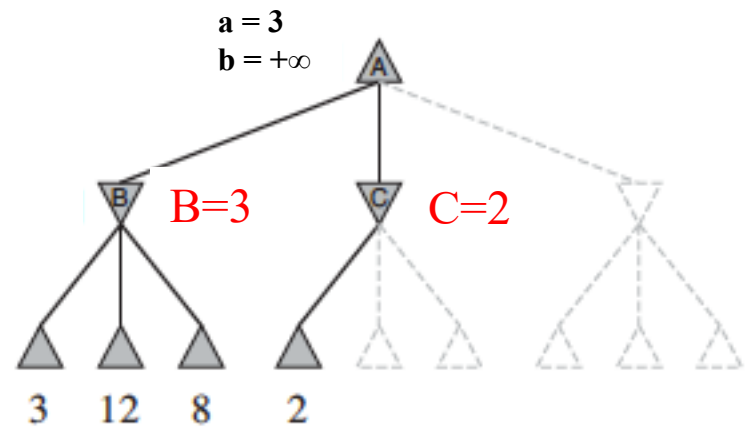
(b)



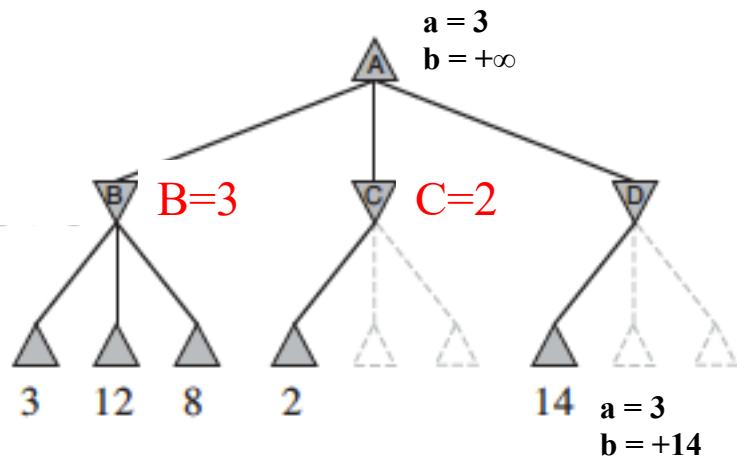
(c)



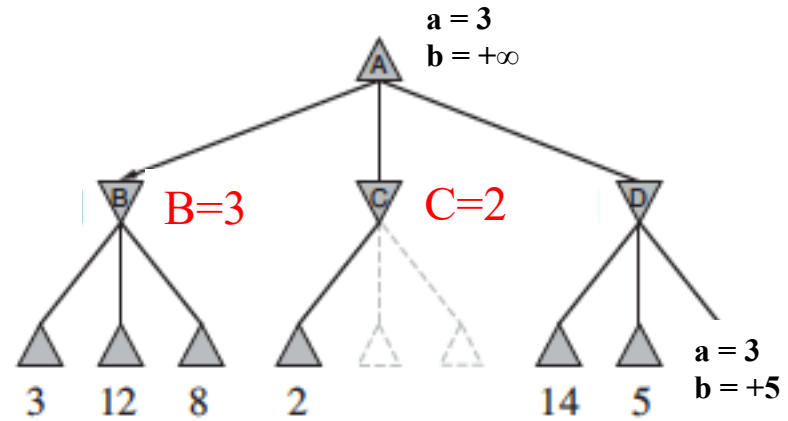
(d)



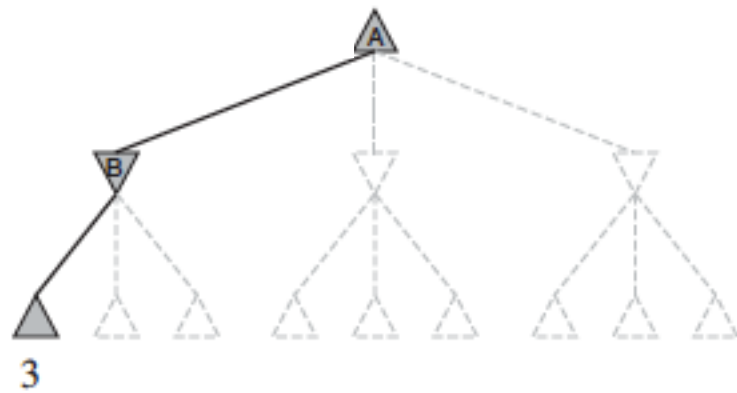
(e)



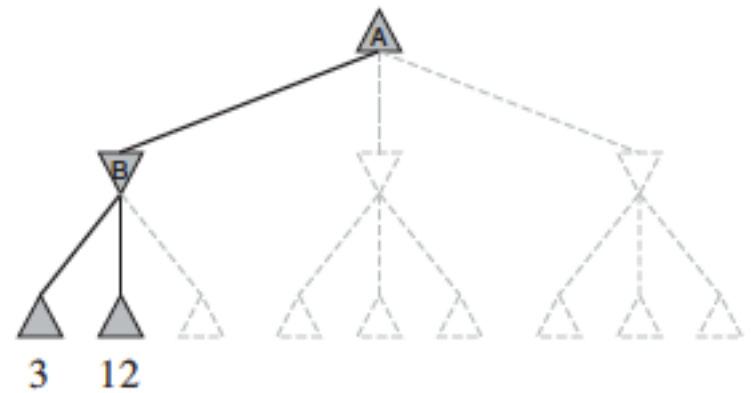
(f)



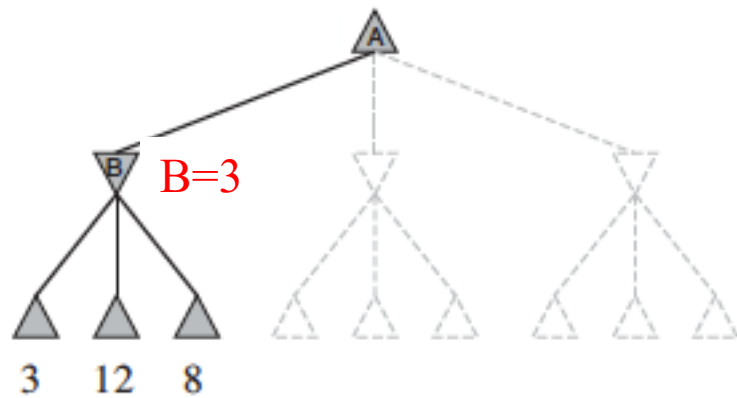
(a)



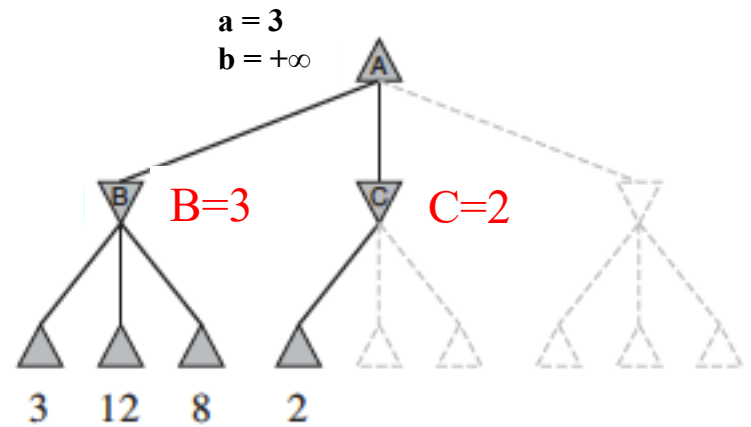
(b)



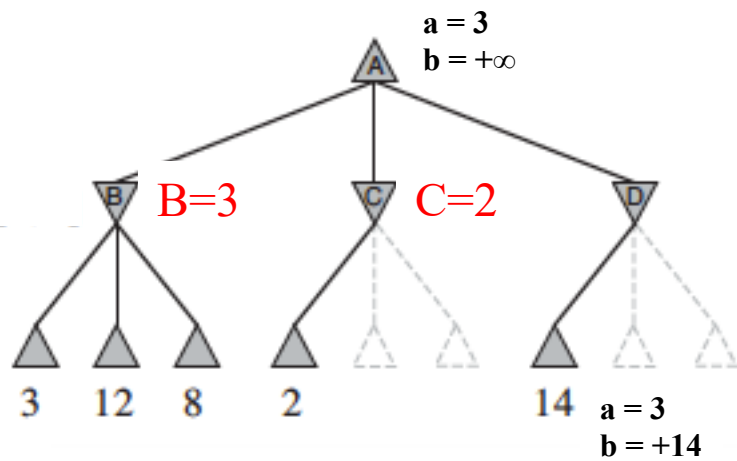
(c)



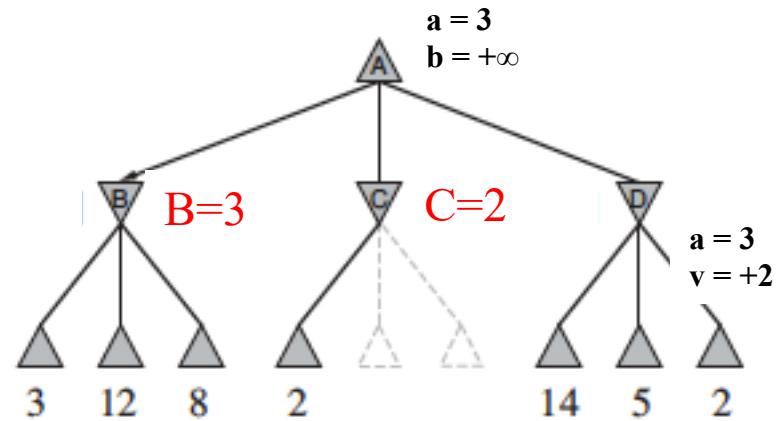
(d)



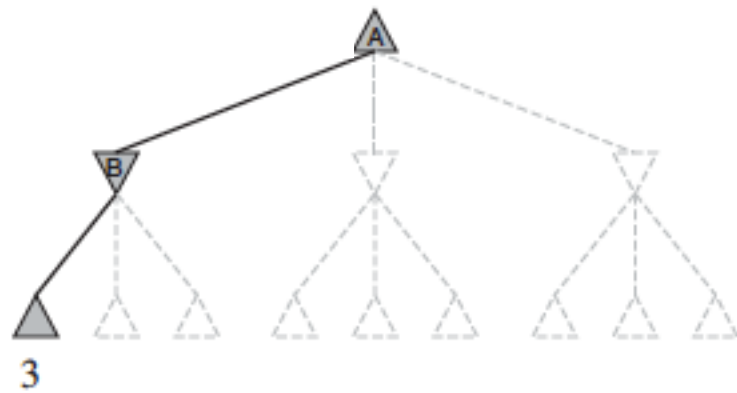
(e)



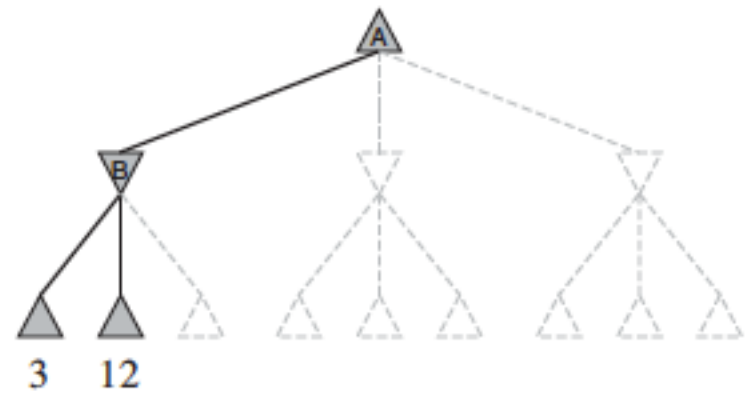
(f)



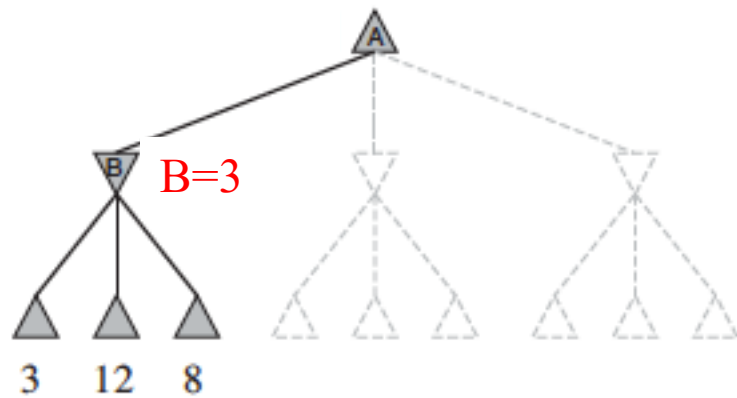
(a)



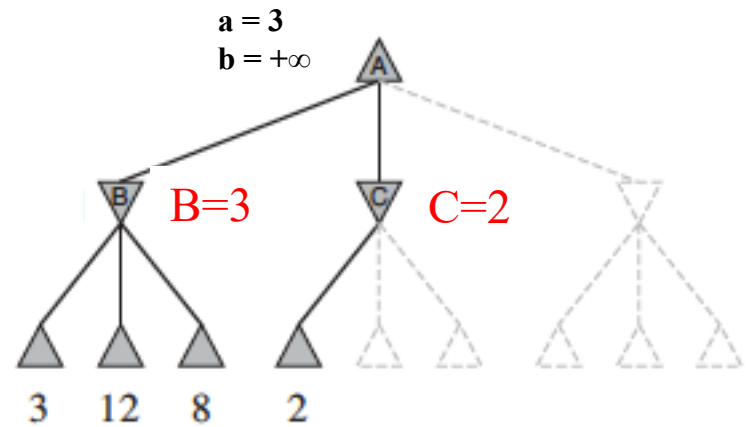
(b)



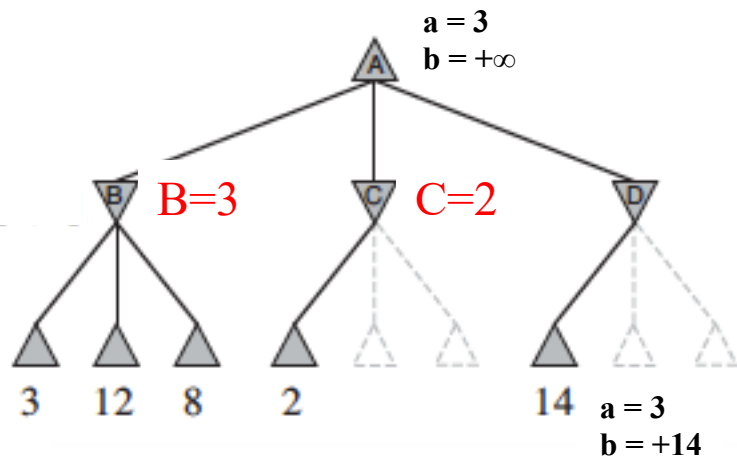
(c)



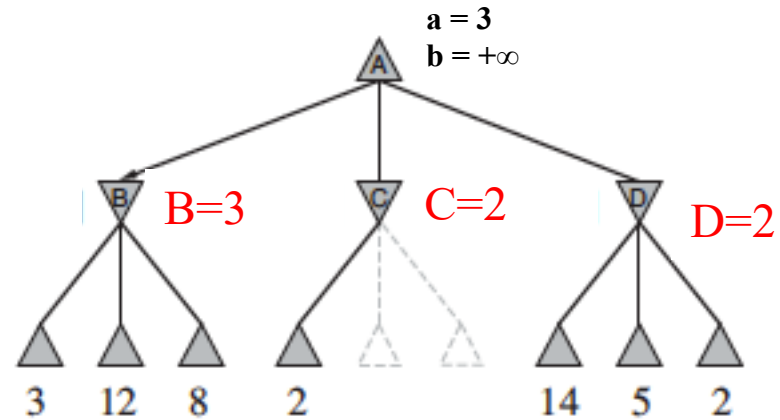
(d)



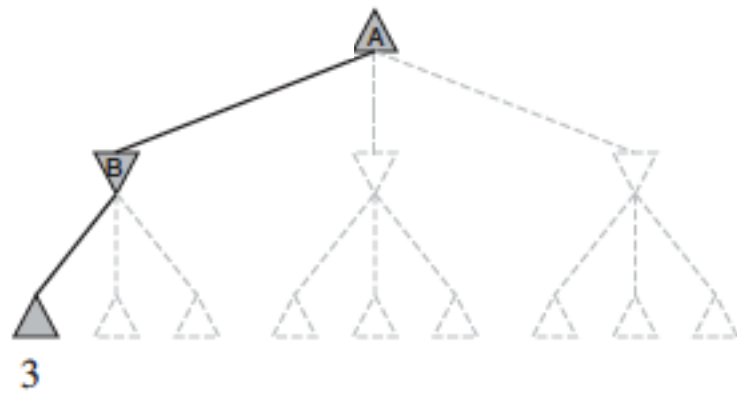
(e)



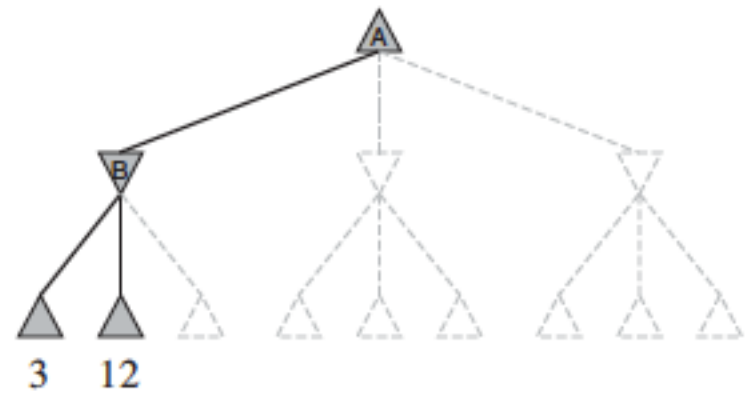
(f)



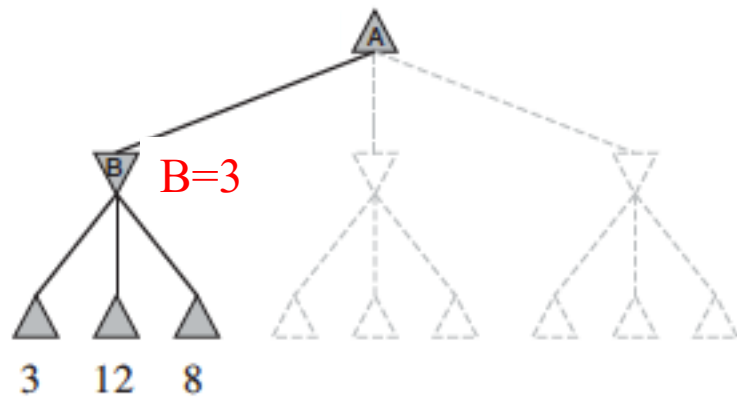
(a)



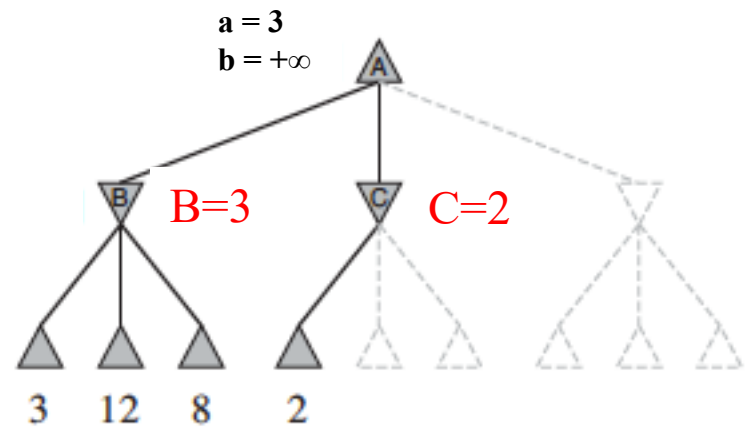
(b)



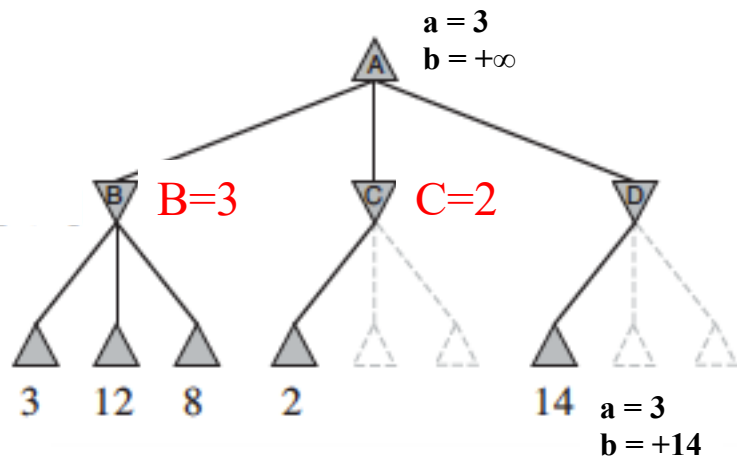
(c)



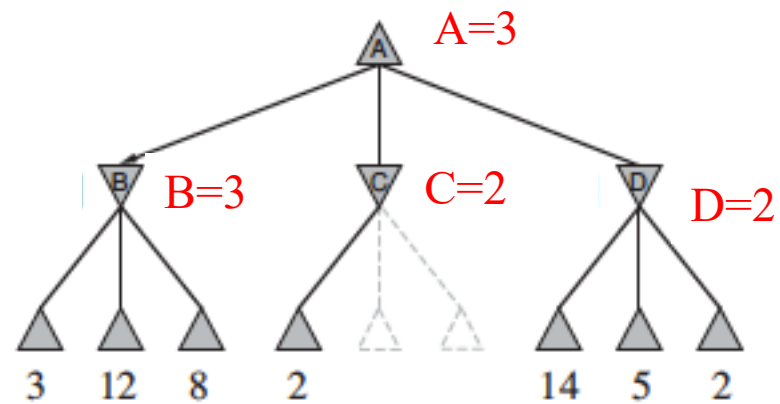
(d)



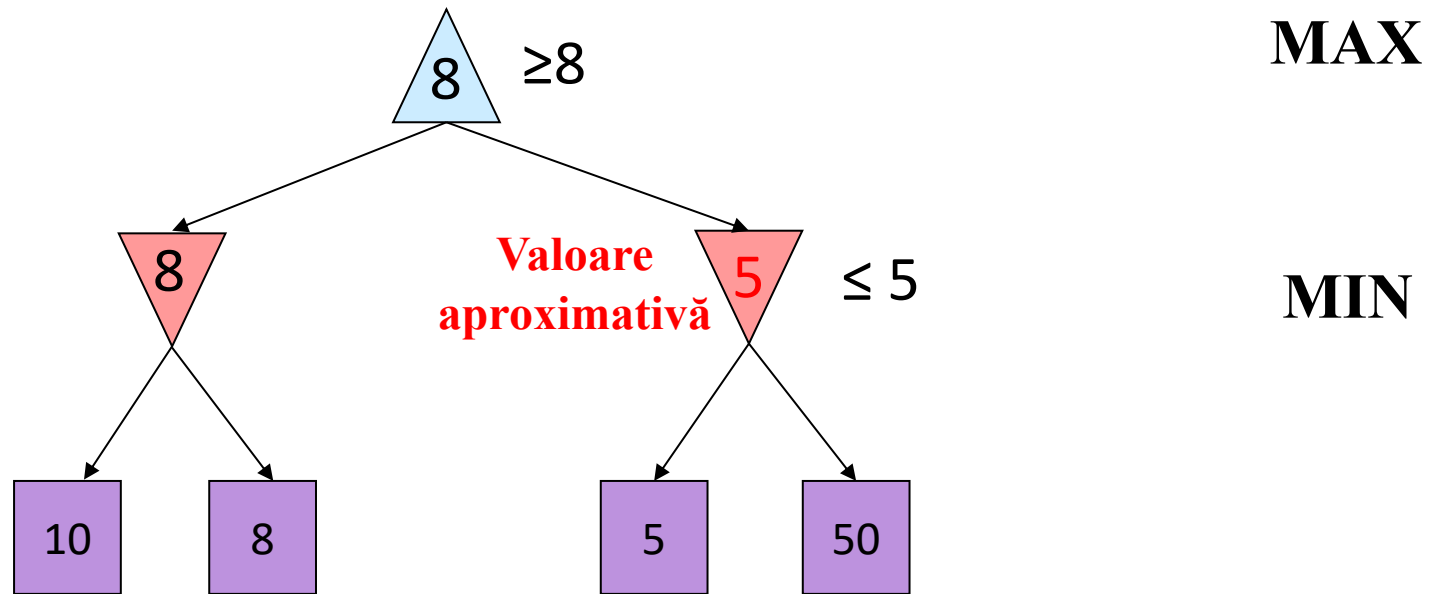
(e)



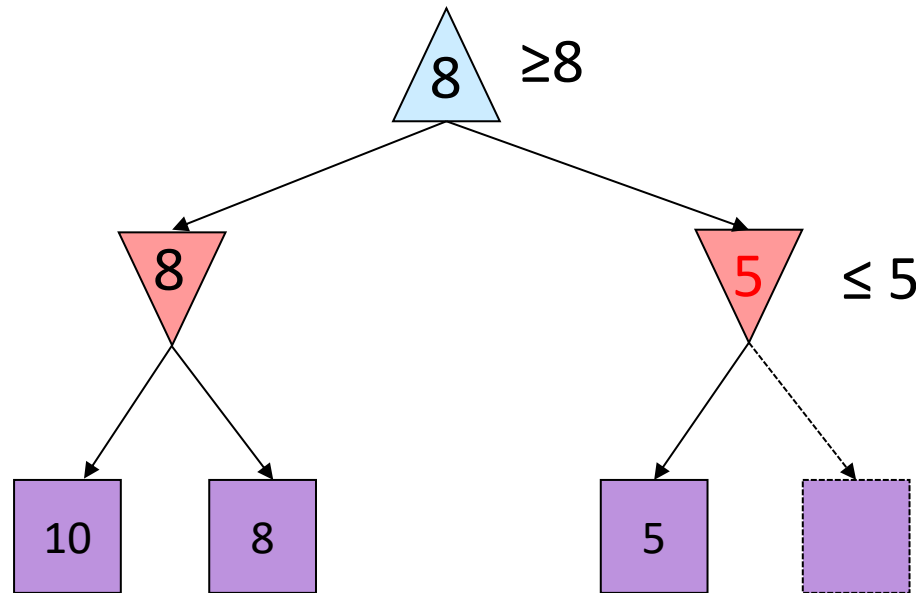
(f)



Exemplu



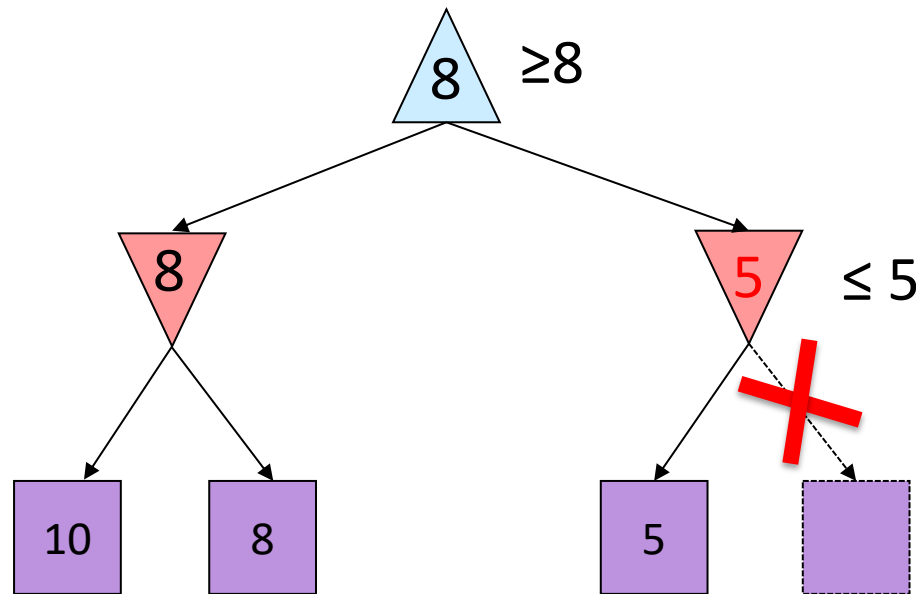
Exemplu



MAX

MIN

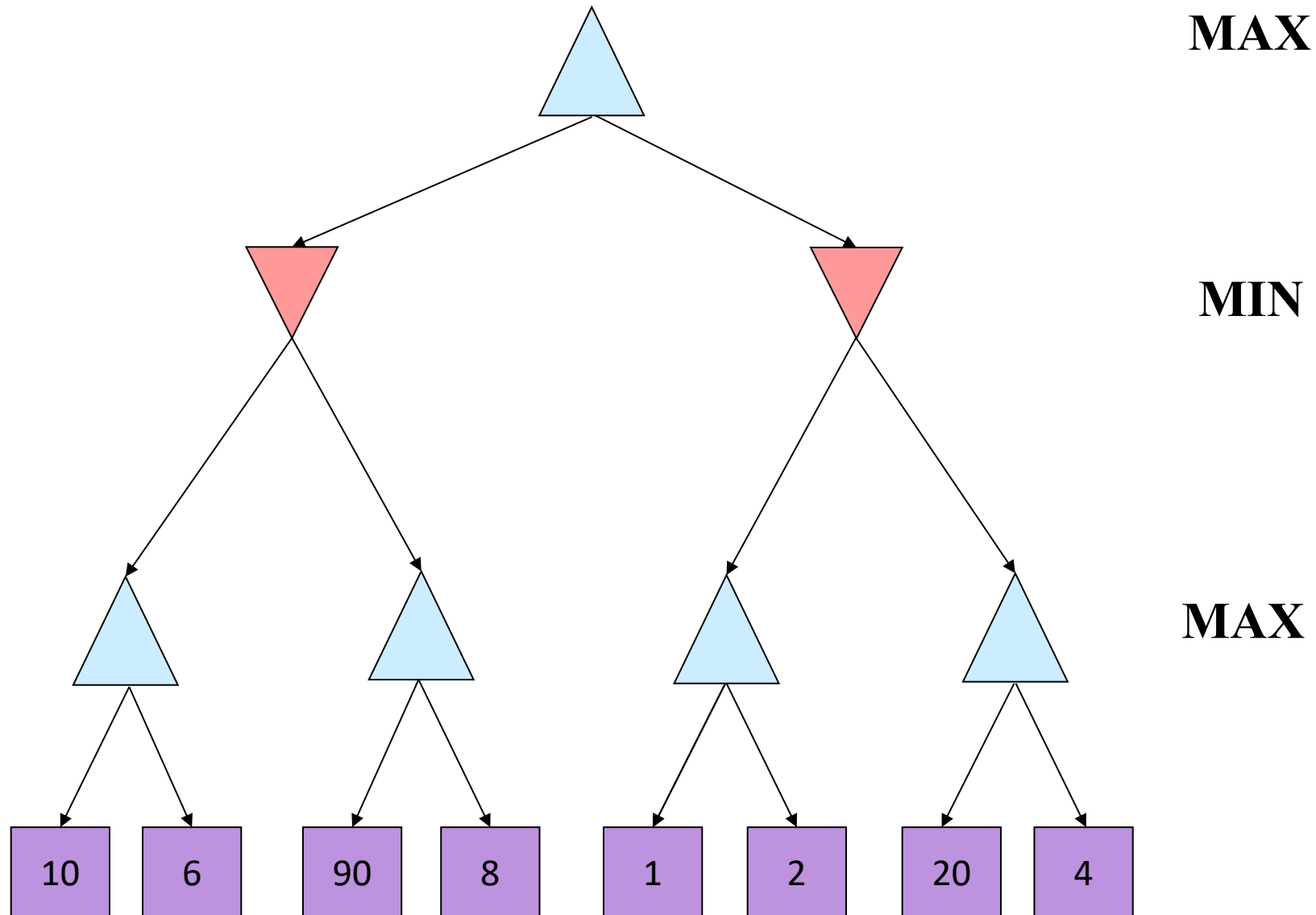
Exemplu



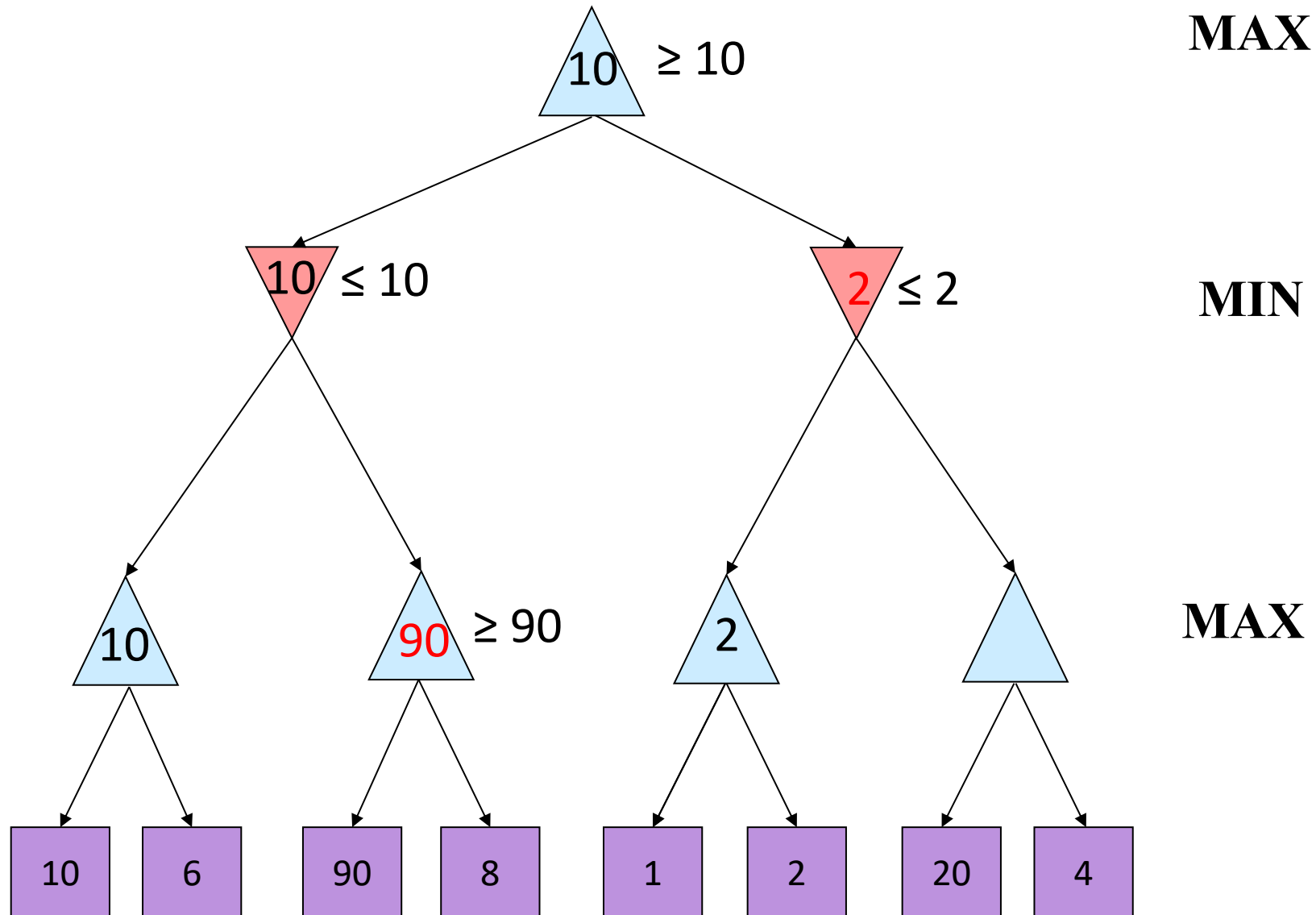
MAX

MIN

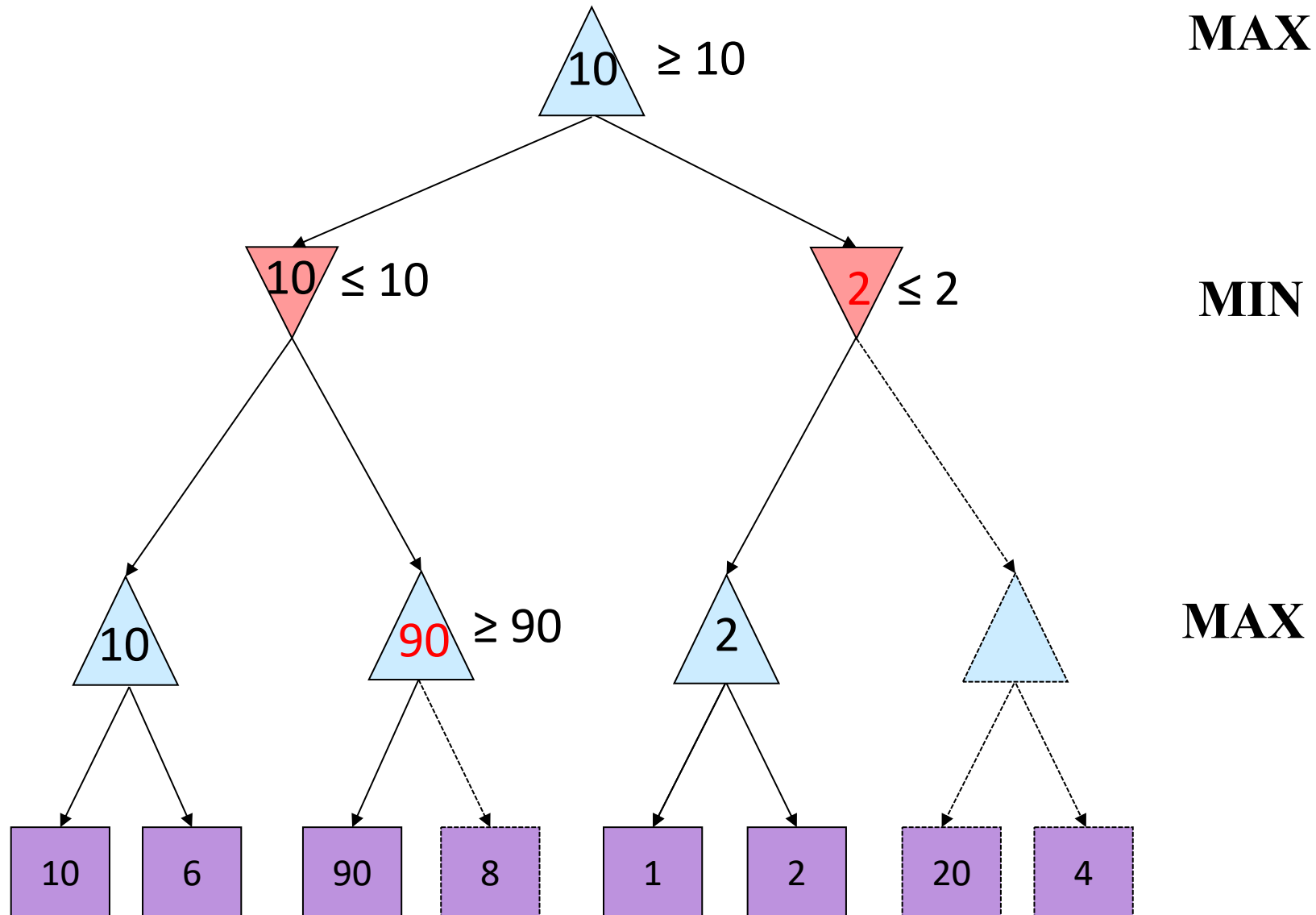
Exemplu



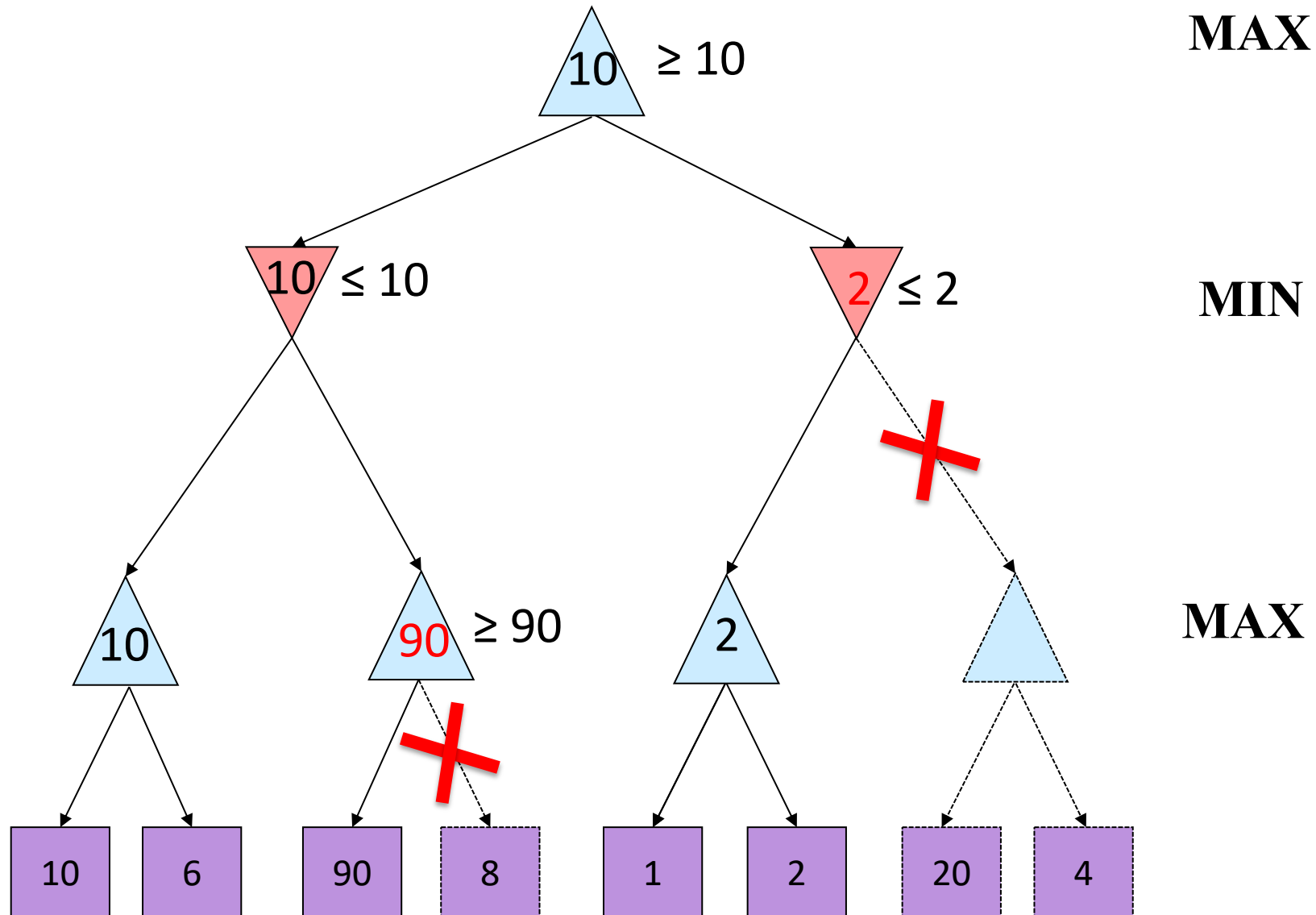
Exemplu



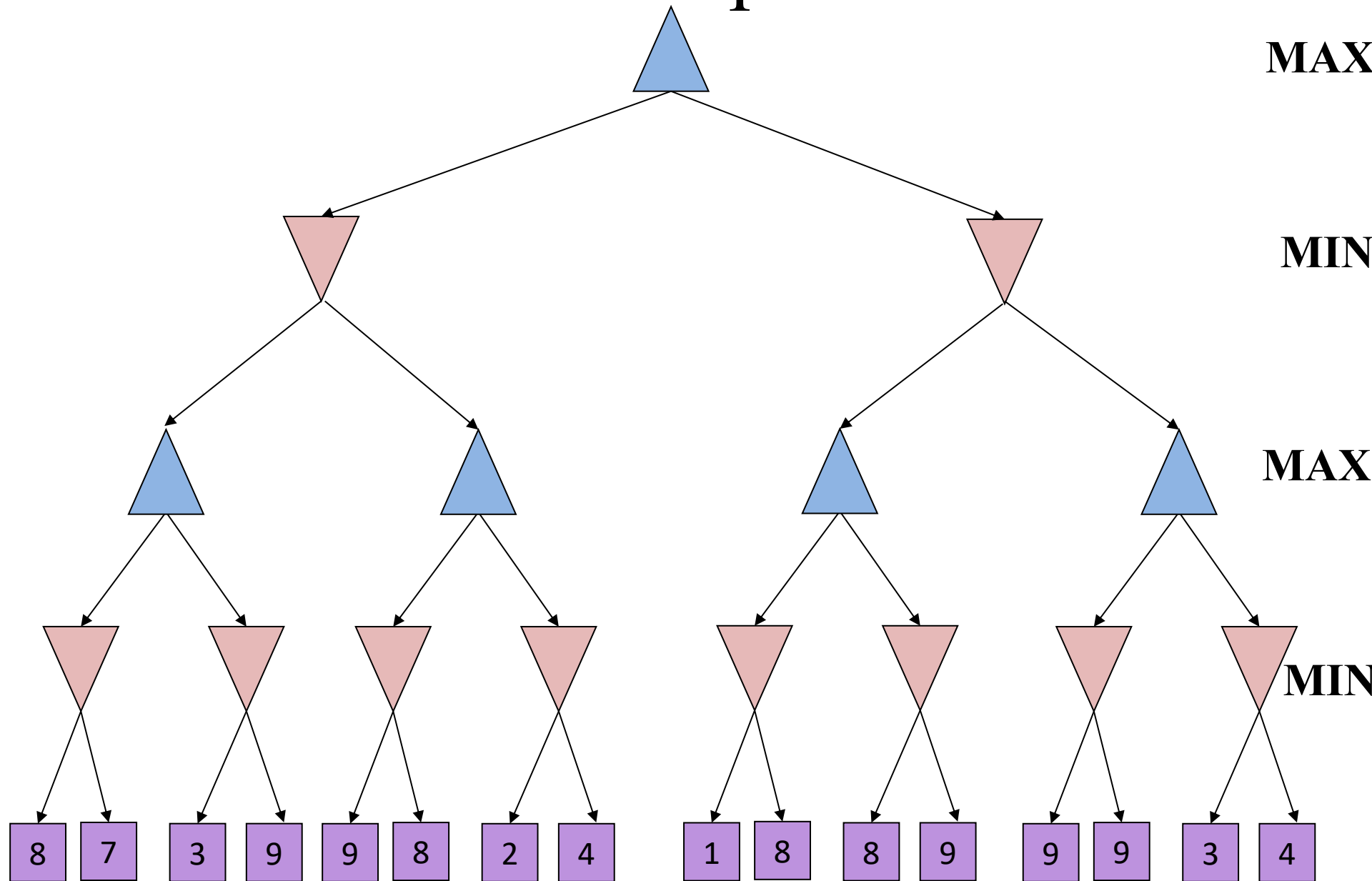
Exemplu



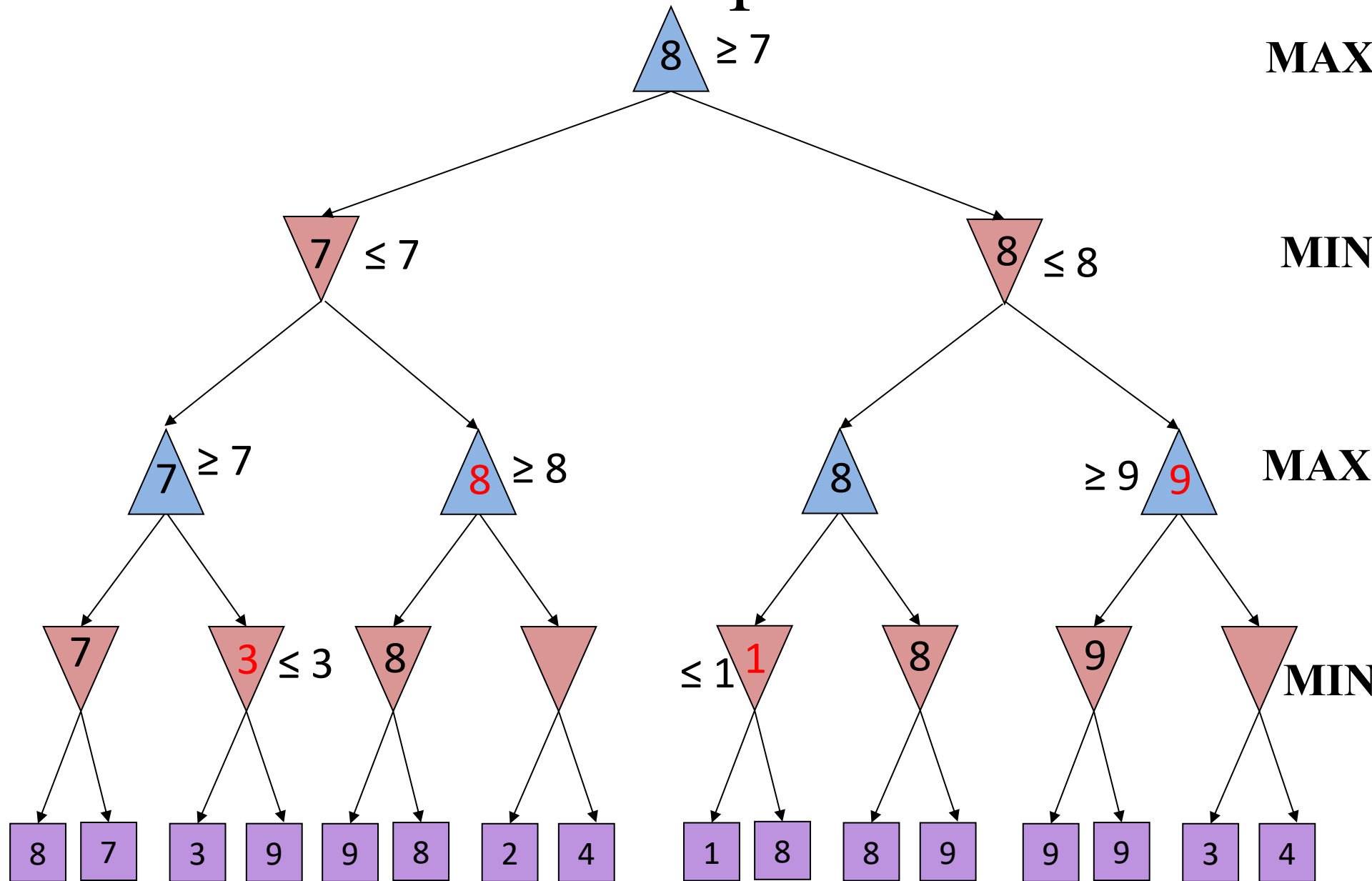
Exemplu



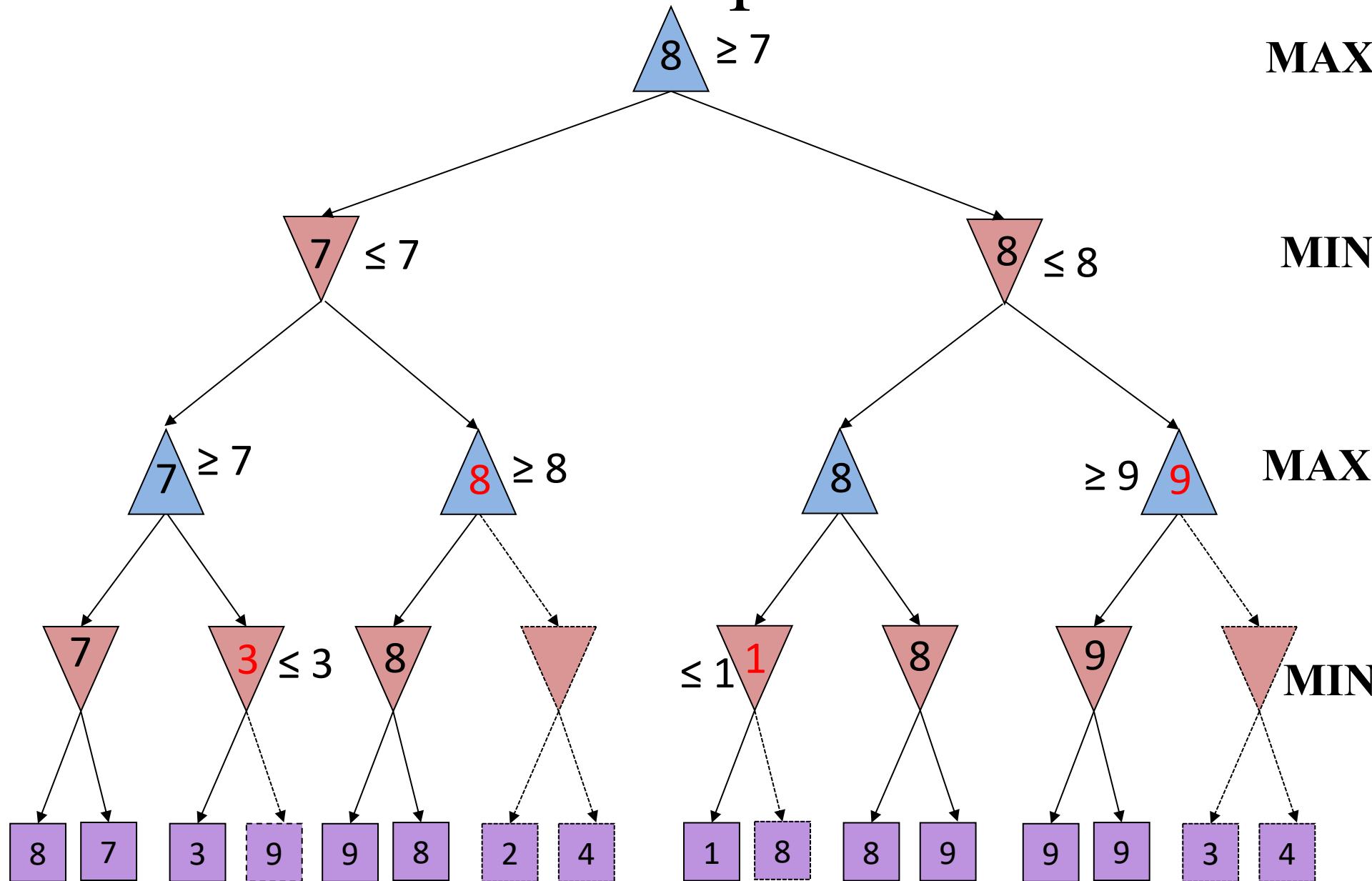
Exemplu



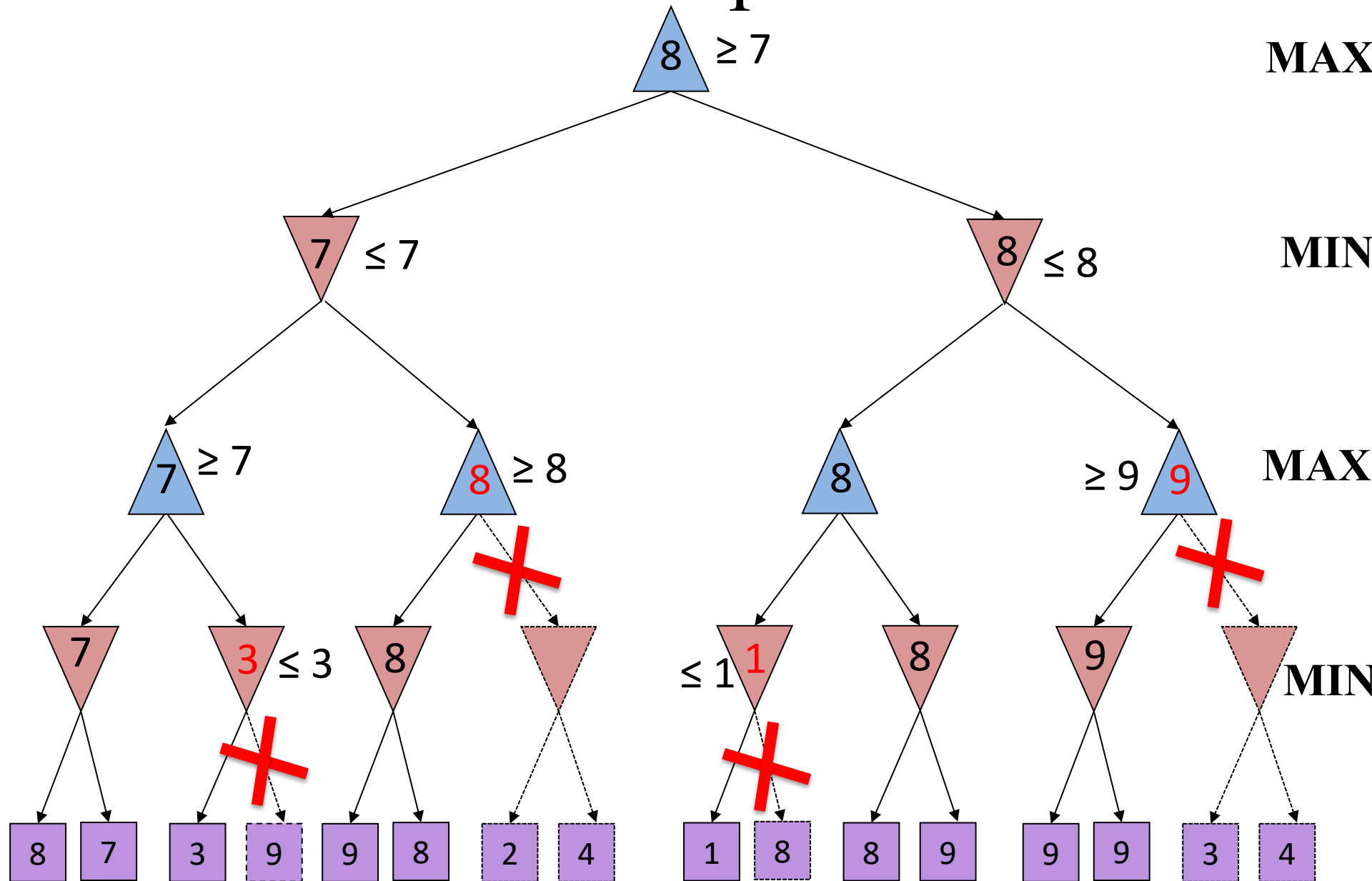
Exemplu



Exemplu



Exemplu



Proprietățile algoritmului Alpha-Beta Pruning

- procesul de retezare nu are niciun efect asupra valorii minimax calculate pentru nodul rădăcină (aceasta nu se schimbă)
- valorile nodurilor intermediare (aproximate) pot fi greșite:
 - important: nodurile fii ale rădăcinii pot avea valori greșiteImplementarea naivă poate conduce la erori
- vizitarea în ordinea bună a nodurilor fii conduce la retezări mai rapide
- cu “ordonare perfectă”:
 - complexitatea timp poate scădea la $O(b^{m/2})$
 - algoritmul poate rula pe o adâncime dublă
 - căutarea exhaustivă nu este totuși posibilă...

